

Image Representation and Filtering

Pablo Mesejo

pmesejo@go.ugr.es

Universidad de Granada

Departamento de Ciencias de la Computación e Inteligencia Artificial



UNIVERSIDAD
DE GRANADA



DaSCI

Instituto Andaluz de Investigación en
Data Science and Computational Intelligence

Index

- What is an image?
- Digital color images
- Introduction to linear filtering and noise reduction
- Cross-correlation and convolution
- Examples of linear filters
- Nonlinear filters, thresholding and histogram equalization

En esta presentación, vamos a introducir a qué nos referimos cuando hablamos de una “imagen”, y vamos también a introducir lo que es una imagen a color, diferenciándola de una imagen en escala de grises. A continuación, vamos a hablar de filtros lineales y su utilidad, por ejemplo, en la reducción del ruido en imágenes. Después, vamos a presentar la diferencia entre correlación cruzada y convolución, y vamos a ver distintos ejemplos con filtros lineales. Finalmente, hablaremos brevemente de filtros no lineales, umbralización y ecualización del histograma de una imagen.

Readings

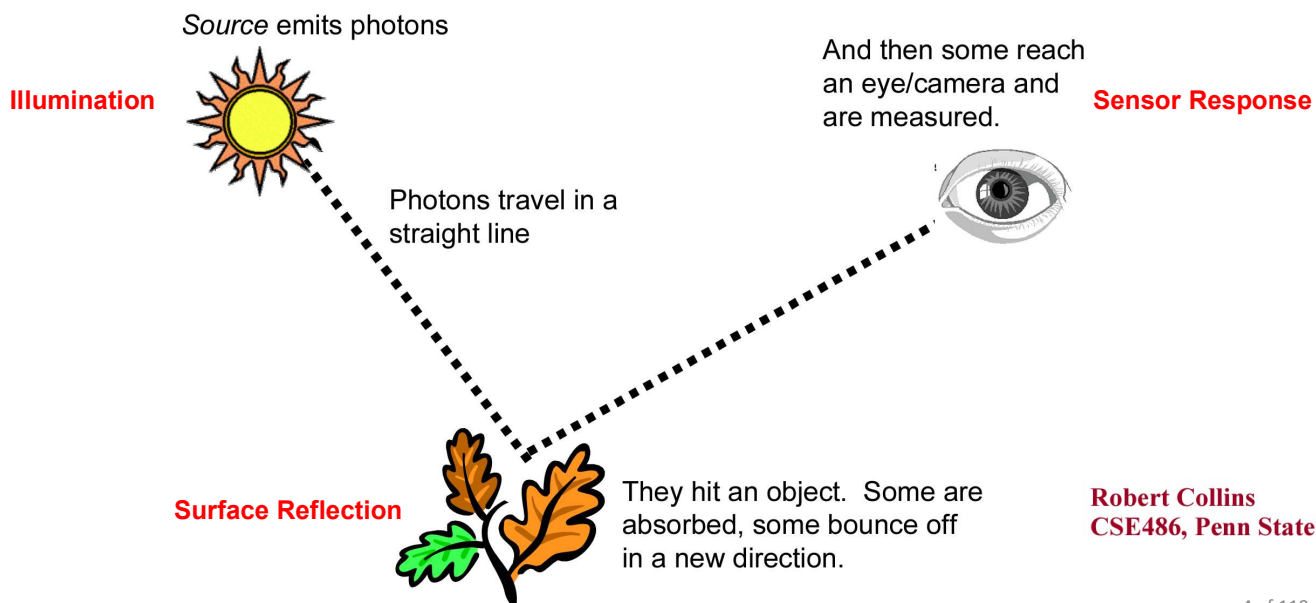
- ***Image Formation & Sampling***
 - Szeliski (2022), Chapter 2
- ***Image Filtering & Edge Detection***
 - Szeliski (2022), Chapter 3 & 7.2
 - Forsyth and Ponce (2012), Chapter 4, 5.1 & 5.2
- Slides credit: many slides have been adapted from those by Noah Snavely, Derek Hoiem and Stan Birchfield, among others.

En esta diapositiva tenéis la bibliografía esencial para este bloque de la asignatura. Como siempre os digo, las explicaciones incluidas en las propias diapositivas, junto con las indicaciones y explicaciones que apporto en clase, deberían ser más que suficientes para comprender la materia.

En cualquier caso, estas referencias se proporcionan tanto para quienes quieran consultar las fuentes originales del material presentado como para aquellos que deseen ampliar sus conocimientos sobre estas temáticas.

Si alguien tiene interés en disponer de más referencias bibliográficas, o recursos de apoyo de todo tipo, tanto en este tema como en cualquier otro, por favor, que no dude en decírmelo.

What is an image?



4 of 119

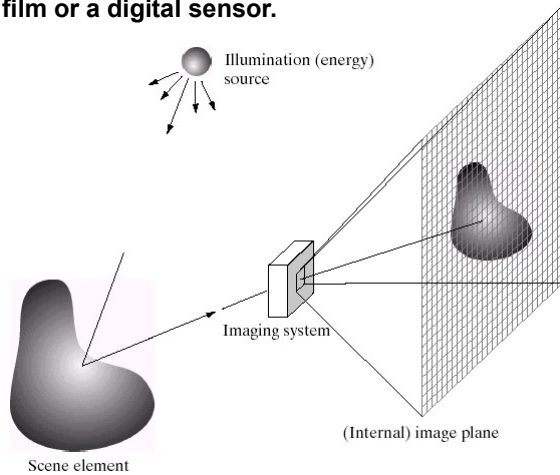
¿Qué es una imagen? Desde un punto de vista físico (y yo no soy un experto en la materia, por lo que puedo caer en alguna simplificación excesiva o imprecisión), podemos partir de una fuente de luz que emite radiación electromagnética, que a nivel cuántico podemos describir en términos de fotones. Un fotón es una excitación del campo electromagnético y presenta ese conocido comportamiento cuántico que, dependiendo del experimento, manifiesta propiedades que asociamos tanto a ondas como a partículas.

En cualquier caso, podemos representar la propagación de la luz mediante rayos que, en un medio homogéneo, siguen líneas rectas. Cuando alcanza la superficie de un objeto, una parte de esa radiación puede ser absorbida y otra parte reflejada o dispersada en distintas direcciones. Parte de la luz que sale de la superficie puede finalmente alcanzar nuestros ojos o el sensor de una cámara, donde es medida y transformada en una señal.

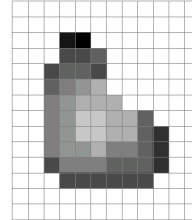
Por tanto, podemos identificar tres elementos fundamentales en el proceso de formación de una imagen: la iluminación, es decir, la luz que llega a la escena; las propiedades de las superficies, que determinan cómo interactúan con esa luz; y la respuesta del sensor, que mide parte de la radiación que finalmente le llega.

What is an image?

A record of a set of light rays that bounce through space and hit a piece of light-sensitive film or a digital sensor.

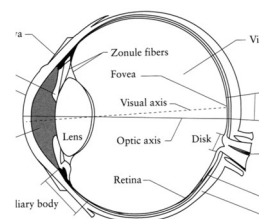


A grid of pixel values.



We focus on this! Digital Images.

A response of our retinas to a light stimulus



We receive two "images" (one for each eye) and our brain assembles the 3D scene.

Source: A. Efros

5 of 119

Podemos definir una imagen desde distintos puntos de vista.

A la izquierda, por ejemplo, podemos entender una imagen como el registro de la luz que atraviesa el espacio y alcanza un sensor digital o una película fotosensible. Aquí estaríamos hablando principalmente del proceso de **formación y captura de la imagen**.

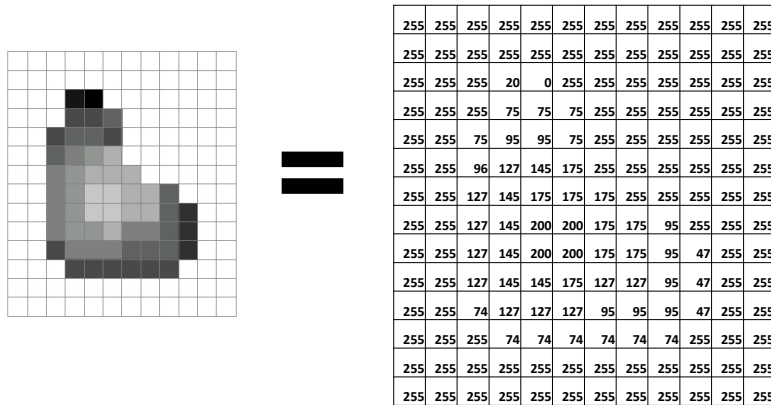
En la parte inferior derecha podemos adoptar un **punto de vista biológico**: una imagen como la respuesta de nuestras retinas ante los estímulos visuales. Recibimos una proyección ligeramente distinta en cada ojo, y nuestro sistema visual integra ambas señales para construir una percepción de la estructura tridimensional de la escena.

Finalmente, podemos utilizar una **definición mucho más sencilla y computacional**, que es la que vamos a emplear en esta asignatura. Una imagen digital no es más que una rejilla o matriz de píxeles, donde cada píxel contiene uno o varios valores numéricos que representan la información registrada en ese punto, por ejemplo la intensidad luminosa o los valores correspondientes a distintos canales de color.

Por tanto, a partir de ahora, cuando hablemos de una imagen, normalmente nos estaremos refiriendo precisamente a esto: una matriz de valores organizada espacialmente.

What is an image?

- A grid (matrix) of intensity (integer) values



(common to use one byte per value: 0 = black, 255 = white)

6 of 119

Y esto es precisamente lo que vemos en esta diapositiva. Como regla general, vamos a interpretar una imagen en escala de grises como una matriz de valores enteros de intensidad.

A la izquierda vemos la representación visual de esa matriz mediante distintas tonalidades de gris y, a la derecha, su representación numérica equivalente. En este caso, el valor 0 se asocia con el negro y el valor 255 con el blanco. Los valores intermedios representan distintas tonalidades de gris: cuanto más próximo esté el valor a 255, más claro será el píxel; y cuanto más próximo esté a 0, más oscuro será.

Esto corresponde al caso habitual de imágenes de 8 bits por píxel, donde disponemos de $2^8 = 256$ niveles posibles de intensidad.

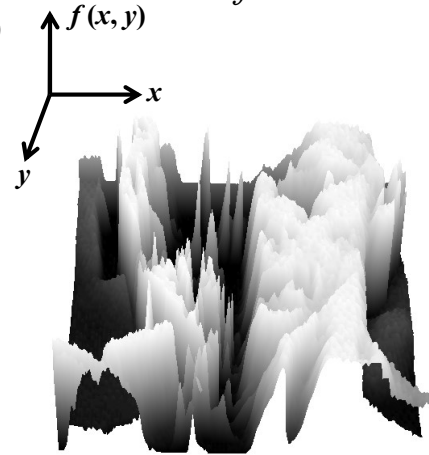
También es importante remarcar que, aunque una imagen suele almacenarse inicialmente como un array de valores enteros, las operaciones de procesamiento que realizamos sobre ella se llevan a cabo en coma flotante. Esto permite representar resultados intermedios con mayor precisión y evita perder información durante los cálculos.

Cuando posteriormente queremos visualizar la imagen, esos valores deben convertirse de nuevo al formato que espera el sistema de representación, normalmente valores enteros dentro de un rango determinado, como 0-255 en una imagen de 8 bits. Para ello, los valores en coma flotante se cuantifican, por ejemplo mediante redondeo, y se ajustan al rango permitido antes de mostrarlos en

pantalla.

What is an image?

- Can think of a (grayscale) image as a **function** f from $\mathbb{R}^2$ to $\mathbb{R}$:
 - $f(x,y)$ gives the **intensity** at position (x,y)



- A **digital** image is a **discrete** (**sampled**, **quantized**) version of this function

We digitize the coordinate values

We digitize the intensities

7 of 119

También podemos pensar en una imagen como una función bidimensional. Es decir, una función que, dada una posición espacial determinada, unas coordenadas (x, y) , nos proporciona el valor de intensidad correspondiente a ese punto.

Esta interpretación nos permite visualizar una imagen como una especie de superficie formada por valles y montañas, donde las coordenadas x e y determinan la posición y la altura representa la intensidad. Por ejemplo, aquí vemos a la izquierda una imagen de *Terminator* y, a la derecha, su representación tridimensional equivalente, en la que la altura de cada punto viene determinada por su nivel de intensidad.

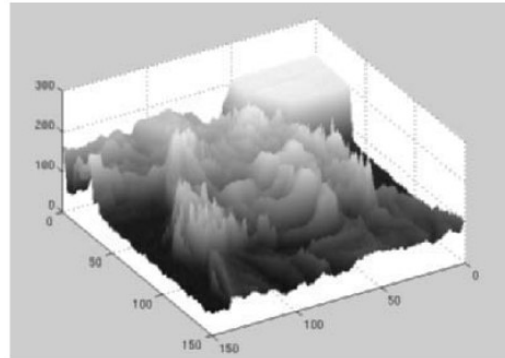
Ahora bien, una imagen digital es una versión discreta de esa función. Para obtenerla se llevan a cabo dos procesos fundamentales: muestreo y cuantización.

El **muestreo** consiste en discretizar el dominio espacial: en lugar de considerar cualquier posible valor continuo de x e y , nos quedamos con un conjunto finito de posiciones, que son precisamente los píxeles de nuestra imagen.

La **cuantización**, por su parte, consiste en discretizar los posibles valores de intensidad. Es decir, en lugar de permitir cualquier valor continuo de intensidad luminosa, lo representamos utilizando un número finito de niveles, por ejemplo los 256 valores comprendidos entre 0 y 255 en una imagen de 8 bits.

What is an image?

- Can think of a (grayscale) image as a **function** f from $\mathbb{R}^2$ to $\mathbb{R}$:
 - $f(x,y)$ gives the **intensity** at position (x,y)



Source:
M. Sonka

- A **digital** image is a **discrete** (**sampled**, **quantized**) version of this function

We digitize the coordinate values

We digitize the intensities

8 of 119

En esta diapositiva vemos, sencillamente, otro ejemplo similar al de la diapositiva anterior. Ambas figuras representan la misma imagen. Pero el segundo modo de visualización (el de la derecha) nos resultará de especial utilidad a la hora de entender cómo operamos con imágenes (incluyendo, por ejemplo, el cálculo de derivadas de una imagen).

What is an image?

– A **digital** image is a **discrete** (**sampled, quantized**) version of this intensity function

- **Sample** the 2D space on a regular grid
- **Quantize** each sample (round to nearest integer)

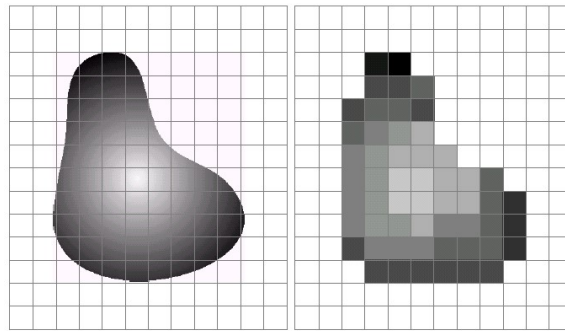


FIGURE 2.17 (a) Continuous image projected onto a sensor array. (b) Result of image sampling and quantization.

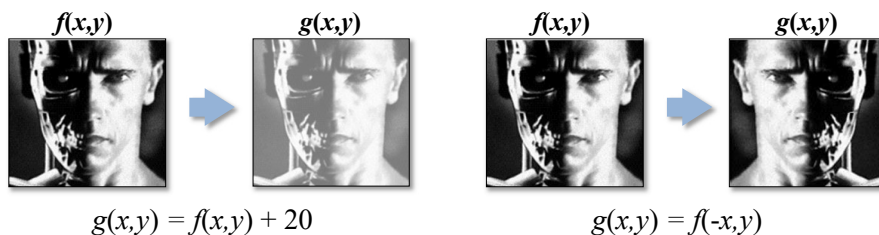
We deal with a **noisy/approximate representation of reality** (inaccurate in terms of intensity and space)

9 of 119

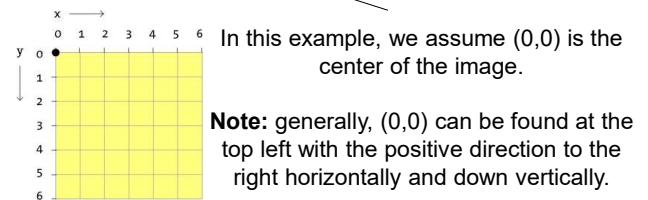
Como consecuencia de lo anterior (muestreo del espacio y cuantización de la intensidad) debemos ser conscientes en todo momento de que operamos con una representación “ruidosa”/aproximada de la realidad.

Image transformations

- As with any function, we can apply (global) operators to an image



- We'll talk about a special kind of operator: **convolution** (local transformation)



10 of 119

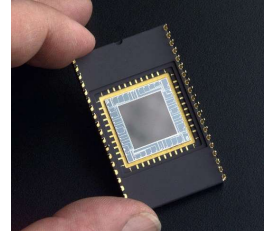
Una de las ventajas de interpretar una imagen como una función es que podemos aplicar sobre ella operadores y transformaciones, tanto globales como locales, de manera análoga a como lo hacemos con otras funciones matemáticas.

Por ejemplo, podemos sumar una cantidad constante a todos los valores de la función, lo que en una imagen en escala de grises supondría incrementar su intensidad y, por tanto, hacerla más clara. También podemos transformar las coordenadas espaciales. Si sustituimos x por $-x$, obtenemos una reflexión especular respecto al eje Y , suponiendo que el origen de coordenadas (0,0) se encuentre en el centro de la imagen. De forma análoga, sustituir y por $-y$ produciría una reflexión respecto al eje X .

Conviene recordar, sin embargo, que en procesamiento digital de imágenes el sistema de coordenadas utilizado habitualmente es diferente del cartesiano convencional: el punto (0,0) suele situarse en la esquina superior izquierda, la coordenada x aumenta hacia la derecha y la coordenada y aumenta hacia abajo.

Un tipo especialmente importante de operador, que veremos más adelante y que resulta fundamental tanto en procesamiento de imágenes como en procesamiento de señales en general, es la convolución. Volveremos sobre ella dentro de unos instantes.

Digital camera



A digital camera replaces film with an image sensor (CCD or CMOS)

- Each cell in the array is a light-sensitive diode that converts photons to electrons.
- The camera collects light that bounces off objects.
- <http://electronics.howstuffworks.com/digital-camera.htm>

11 of 119

Una cámara digital sustituye la película fotosensible por una matriz de elementos fotosensibles, cuyo conjunto se denomina habitualmente sensor. Cada uno de esos elementos convierte la energía de los fotones que recibe en una señal eléctrica.

La cantidad de señal generada depende de la cantidad de luz que alcanza cada elemento fotosensible. Posteriormente, esa señal eléctrica se digitaliza y da lugar a los valores numéricos que constituyen la imagen.

En resumen, la cámara recoge la luz reflejada por los objetos de la escena y la transforma en una matriz de valores digitales.

Digital color images

Color images,
RGB color
space



R



G



B

Three gray
level images

12 of 119

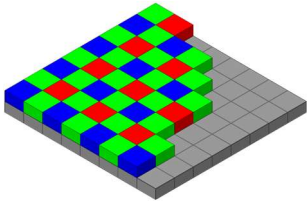
En una imagen RGB, cada píxel ya no contiene un único valor de intensidad, sino tres valores, correspondientes a los canales rojo, verde y azul.

Por tanto, podemos interpretar una imagen en color como tres imágenes en escala de grises, una por cada canal. En cada una de ellas, los valores más altos indican una mayor contribución de ese color en esa posición.

La combinación de los tres canales R , G y B da lugar al color final que observamos en cada píxel.

Digital color images

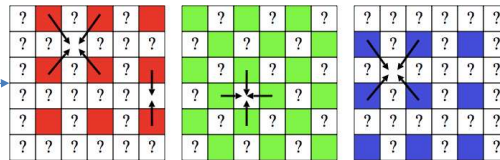
Typically, each 2x2 block is encoded as RGGB



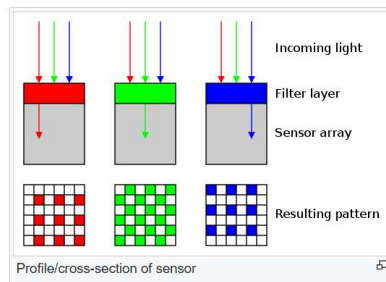
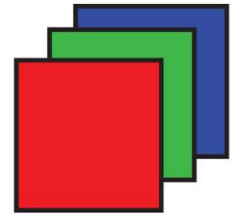
The **Bayer Filter** is one color filter array (patented in 1976).

Green: 50%, Red: 25%, and Blue: 25%

Mimics the physiology of the human eye (M cones are most sensitive to green light, whereas L cones peak in the yellow-green region).



RGB image is computed by interpolation



https://en.wikipedia.org/wiki/Bayer_filter

This strategy is much less expensive than a three-CCD camera

(https://en.wikipedia.org/wiki/Three-CCD_camera)

Very interesting video on Color Perception (by Steve Seitz):

<https://www.youtube.com/watch?v=cM2wvHtVcM8>

13 of 119

Una posibilidad para capturar imágenes en color es utilizar tres sensores independientes, uno para cada componente roja, verde y azul. Este sistema se ha utilizado históricamente en cámaras profesionales, pero resulta más complejo y costoso.

Una solución mucho más habitual consiste en utilizar un único sensor, CCD o CMOS, cubierto por un filtro de Bayer. En este filtro, cada fotosensor mide únicamente una componente de color: rojo, verde o azul. Típicamente, en cada bloque de 2×2 tenemos dos sensores verdes, uno rojo y uno azul.

Como en cada posición solo conocemos directamente uno de los tres canales, los otros dos se estiman a partir de los píxeles vecinos mediante un proceso denominado demosaicing o interpolación cromática. Así obtenemos finalmente los tres canales RGB completos.

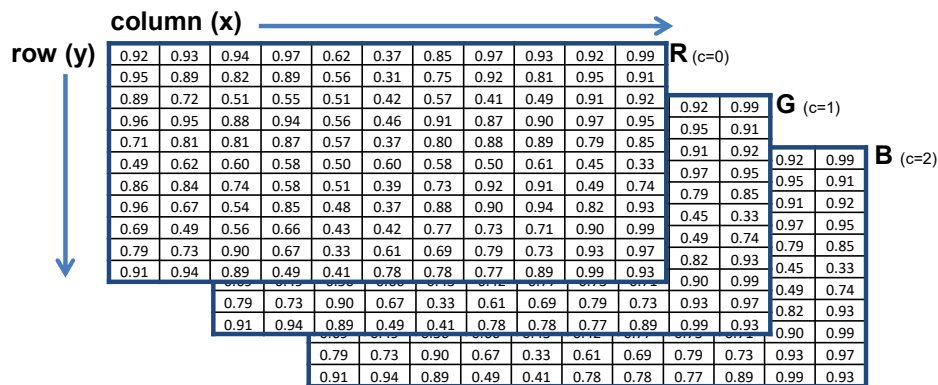
Hay el doble de sensores/filtros verdes porque el sistema visual humano es especialmente sensible a la información de luminancia ($\approx$ "cómo de claro/brillante vemos un punto"), a la que el canal verde contribuye de manera importante. Esta distribución permite obtener una buena calidad de imagen utilizando un único sensor y, por tanto, con un coste mucho menor.

Digital color images in Python

```
im = cv2.imread(filename) # read image
im = cv2.cvtColor(im, cv2.COLOR_BGR2RGB) # order channels as RGB
im = im / 255 # values range from 0 to 1
```

- RGB image `im` is a `H x W x 3` matrix (numpy.ndarray)
- `im[0,0,0]` = top-left pixel value in R-channel
- `im[y, x, c]` = pixel at row `y`, column `x`, channel `c`
- `im[H-1, W-1, 2]` = bottom-right pixel in B-channel

Very important to operate with floating-point numbers!



Slide credit:
Derek Hoiem

14 of 119

En esta diapositiva aparece algo que vais a ver con frecuencia a lo largo del curso: incluso en las clases de teoría introduciré pequeñas notas técnicas. Para mí, comprender bien una materia implica conocer tanto sus fundamentos teóricos como la forma en que esos conceptos se materializan en la práctica. Teoría y práctica van continuamente de la mano.

En este caso vemos cómo podemos trabajar en Python con una imagen digital en color utilizando OpenCV. En la primera línea importamos el paquete `cv2` y utilizamos la función `imread` para leer el fichero indicado por `filename`. La imagen resultante queda almacenada en la variable `im`.

Por defecto, OpenCV almacena las imágenes en orden BGR, en lugar del orden RGB que solemos utilizar. Por ello, en la segunda línea reordenamos los canales para pasar de BGR a RGB.

A continuación, dividimos los valores de la imagen por 255. Las imágenes leídas tienen valores enteros de tipo `uint8` comprendidos entre 0 y 255. Al dividir por 255 obtenemos valores en punto flotante en el intervalo `[0,1]`, lo que resulta conveniente para la mayor parte de operaciones posteriores.

Una imagen RGB puede representarse entonces como un array de NumPy tridimensional de tamaño `H x W x 3`, donde `H` es la altura, `W` el ancho y la tercera dimensión corresponde a los tres canales de color.

Por ejemplo, `im[0,0,0]` representa el valor del canal rojo del píxel situado en la esquina superior izquierda. De forma general, `im[y,x,c]` accede al píxel situado en la

fila y , y la columna x , dentro del canal c . Al aumentar y nos desplazamos hacia abajo en la imagen y al aumentar x nos desplazamos hacia la derecha.

Finalmente, $im[H-1,W-1,2]$ representa el valor del canal azul del píxel situado en la esquina inferior derecha. Hay que recordar que, como ocurre habitualmente en Python, los índices comienzan en 0.

Filters

- Filtering
 - **Form a new image whose pixel values are a combination of the original pixel values**
- Why?
 - To get useful information from images
 - E.g., extract edges (to understand shape)
 - To enhance images
 - E.g., to remove noise
 - E.g., to sharpen details
 - To detect patterns
 - Template matching
 - A key concept/operator in Convolutional Neural Networks

15 of 119

Ahora vamos a empezar a ocuparnos de los filtros y del filtrado de imágenes. De forma general, entenderemos por filtrado la creación de una nueva imagen cuyos valores de píxel se obtienen a partir de una combinación de los valores de la imagen original.

¿Para qué sirve el filtrado? Tiene múltiples aplicaciones. Una de ellas es extraer información relevante de la imagen; por ejemplo, podemos utilizar determinados filtros para detectar bordes y obtener información sobre la forma y la estructura de los objetos.

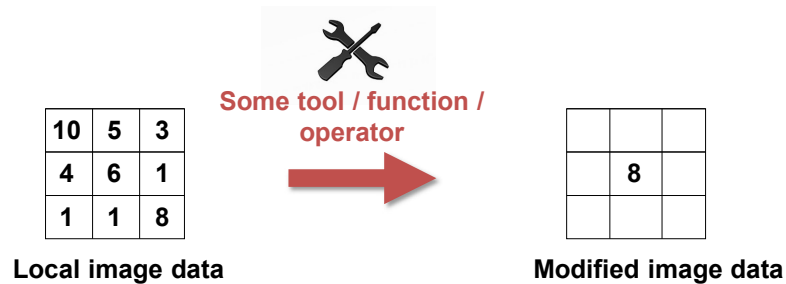
Otra aplicación es la mejora y procesamiento de imágenes. Podemos utilizar filtros para reducir ruido o, por el contrario, realzar determinados detalles.

El filtrado también puede utilizarse para detectar patrones. Un ejemplo clásico es el template matching, donde buscamos en una imagen regiones similares a un patrón o plantilla determinada.

Finalmente, el filtrado es también un concepto fundamental en las redes neuronales convolucionales. Las capas convolucionales aplican filtros aprendidos automáticamente a los datos de entrada para extraer progresivamente distintas características de la imagen.

Image filtering

- We generally modify the pixels in an image based on some function of a **local neighborhood** around each pixel



16 of 119

En esta diapositiva vemos un ejemplo muy sencillo de filtrado de una imagen. Como comentábamos, la idea consiste en generar una nueva imagen modificando los valores de los píxeles de la imagen original.

En muchos casos, este proceso se realiza aplicando una función u operador sobre un vecindario local alrededor de cada píxel. Es decir, el nuevo valor de un píxel no depende únicamente de su valor original, sino también de los valores de los píxeles que lo rodean.

En el ejemplo de la diapositiva tenemos un vecindario de 3x3 y aplicamos sobre él algún operador local. Como resultado, el píxel central, que inicialmente tenía valor 6, pasa a tener valor 8. Todavía no nos preocupa cuál es exactamente el operador utilizado; lo importante en este punto es entender la idea de que el nuevo valor se obtiene a partir de la información contenida en un entorno local del píxel.

Image filtering

- We generally modify the pixels in an image based on some function of a **local neighborhood** around each pixel
 - Wait... why a local neighborhood?? Why not global or individual??? 🤔
 - Nearby pixels show dependence between their values.
 - Weaker and less predictable relationships between distant pixels/regions (high spatial variability).
 - The isolated and decontextualized analysis of the pixels is worthless. Local information is essential.



Pixels, by themselves, are not relevant. What is relevant is the relationship among pixels.

17 of 119

Aquí es donde puede surgir una pregunta natural: ¿por qué utilizamos normalmente un vecindario local? ¿Por qué no analizamos cada píxel de manera completamente aislada o, por el contrario, toda la imagen de forma global?

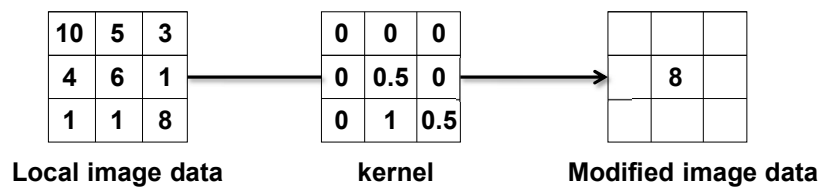
El motivo principal es que, en las imágenes naturales, los píxeles cercanos suelen presentar una fuerte dependencia estadística. Regiones próximas tienden a compartir propiedades como intensidad, color, textura o pertenencia a un mismo objeto. En cambio, a medida que aumenta la distancia espacial, esa relación suele ser menos directa y mucho más variable, aunque pueden existir también ciertas dependencias de largo alcance.

Por otro lado, en general, analizar cada píxel de forma aislada y descontextualizada proporciona muy poca información: el contexto local resulta fundamental.

La diapositiva ilustra esto último de una forma muy sencilla. A la izquierda vemos la imagen de una cebra y, a la derecha, se muestran exactamente los mismos valores de píxel después de haber sido reordenados aleatoriamente. Es decir, se conserva el mismo conjunto de píxeles, pero se destruye por completo su organización espacial. Al perderse las relaciones locales entre ellos, la estructura de la imagen desaparece y la cebra se vuelve irreconocible.

Linear filtering

- One simple version of filtering: linear filtering
 - Replace each pixel by a linear combination (a weighted sum) of its neighbors (cross-correlation, convolution)
- The weights for the linear combination are called the “kernel” / “mask” / “filter”



Kind reminder:

$$L \text{ is a linear operator iff } L(af + bg) = aL(f) + bL(g)$$

↑ ↑
Scalars
↑ ↑
Images

18 of 119

En esta diapositiva retomamos el ejemplo de hace dos diapositivas, con el vecindario de 3×3 píxeles, y nos introducimos ya en el filtrado lineal. En este tipo de filtrado, cada píxel de la imagen de salida se obtiene como una combinación lineal, es decir, una suma ponderada, de los píxeles de un vecindario de la imagen de entrada. A partir de aquí podremos introducir dos operaciones estrechamente relacionadas: la correlación cruzada y la convolución.

Los pesos utilizados para realizar esta combinación lineal se organizan en una pequeña matriz que solemos denominar kernel, máscara o filtro. En nuestro ejemplo, anteriormente simplemente veíamos que el píxel central pasaba de tener un valor 6 en la imagen original a un valor 8 en la imagen filtrada. Ahora podemos ver cómo se obtiene ese 8 utilizando el kernel mostrado en el centro de la diapositiva:

$$10 \cdot 0 + 5 \cdot 0 + 3 \cdot 0 + 4 \cdot 0 + 6 \cdot 0.5 + 1 \cdot 0 + 1 \cdot 0 + 1 \cdot 1 + 8 \cdot 0.5 = 8.$$

Es decir, multiplicamos cada píxel del vecindario por el peso correspondiente del kernel y sumamos todos los resultados.

¿Por qué hablamos de filtrado lineal? Porque el operador de filtrado L cumple la propiedad de linealidad:

$$L(a f + b g) = a L(f) + b L(g),$$

donde f y g son imágenes, y a y b son escalares. Esto implica dos propiedades: aditividad, $L(f + g) = L(f) + L(g)$, y homogeneidad o escalado, $L(a f) = a L(f)$.

Common types of noise

- **Impulse noise:** random occurrences of pixels with values very different from their surroundings.
 - **Salt-and-pepper noise:** random occurrences of both bright and dark pixels.
 - **Salt noise:** random occurrences of bright pixels only.
- **Gaussian noise:** additive intensity variations sampled from a Gaussian (normal) distribution.
- Different noise models require different filtering strategies.



Original



Salt-and-pepper noise



Salt noise



Gaussian noise

Slide adapted from S. Seitz. 19 of 119

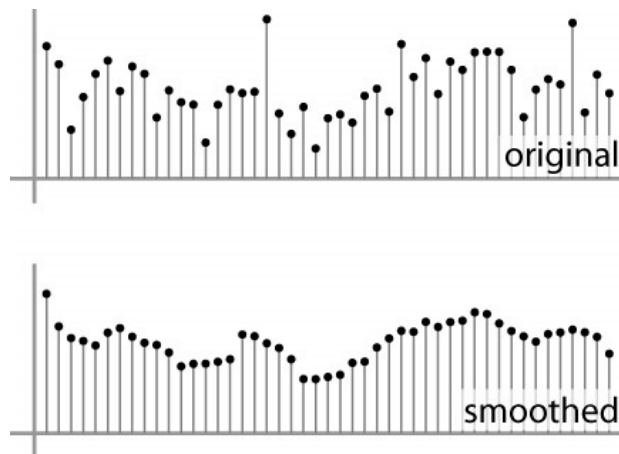
En esta diapositiva vemos algunos tipos de ruido habituales en imágenes. El ruido impulsivo se caracteriza por la aparición aleatoria de píxeles con valores muy diferentes de los de su entorno. Un caso particular es el ruido de sal y pimienta, en el que aparecen de forma aleatoria píxeles muy claros y muy oscuros. Si esos impulsos corresponden únicamente a valores altos de intensidad, hablamos de ruido de sal.

Otro caso muy común es el ruido gaussiano, en el que a la intensidad original de los píxeles se añaden perturbaciones aleatorias que siguen una distribución normal o gaussiana.

Lo importante aquí es entender que distintos modelos de ruido tienen características distintas y, por tanto, requieren estrategias de filtrado diferentes. Si conocemos o podemos aproximar el tipo de ruido presente en la imagen, podemos seleccionar métodos de reducción de ruido más adecuados y evitar, en la medida de lo posible, la pérdida de información relevante.

Smoothing for noise reduction

- Let's replace each pixel with an average of all the values in its neighborhood
- Moving average in 1D:



Source: S. Marschner

20 of 119

Ahora veamos un ejemplo muy sencillo de reducción de ruido utilizando una señal 1D.

La idea consiste en sustituir cada valor de la señal por el promedio de los valores de un pequeño vecindario alrededor de esa posición. Es decir, cada nuevo valor se obtiene combinando la información local de los puntos próximos mediante una media.

En la parte superior de la diapositiva vemos la señal original, que presenta variaciones rápidas asociadas al ruido. En la parte inferior aparece la señal resultante después de aplicar este filtro de medias. Como puede observarse, la señal queda más suave porque las variaciones locales bruscas se atenúan.

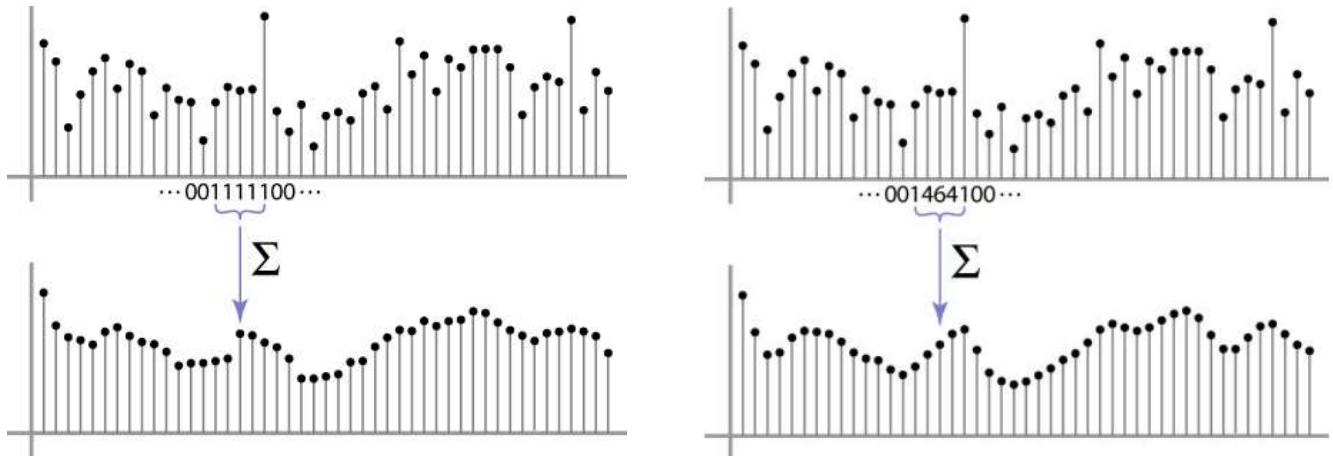
Eso sí, este suavizado tiene un coste: además de reducir ruido, también puede eliminar o difuminar detalles reales de la señal.

Smoothing for noise reduction

- Can add weights to our moving average. For instance, non-uniform weights

$[1, 4, 6, 4, 1] / 16$

What is the shape of these weights? 🤔
Does it look familiar?



21 of 119

Evidentemente, a una media móvil podemos asignarle pesos diferentes. No es necesario que todos los valores del vecindario contribuyan por igual al nuevo valor de la señal.

En la parte izquierda de la diapositiva vemos un filtro de medias uniforme: todos los elementos del vecindario tienen el mismo peso. Si utilizamos cinco valores, cada uno contribuye de la misma manera y, para obtener realmente una media, dividimos la suma entre 5.

En la parte derecha utilizamos, en cambio, una media ponderada con pesos 1, 4, 6, 4 y 1. En este caso damos mayor importancia a los valores próximos al centro del vecindario y, especialmente, al valor central, cuyo peso es 6. Para mantener la escala de la señal, estos pesos se normalizan dividiendo por su suma, que en este caso es 16.

La idea es que los puntos más cercanos a la posición que estamos estimando suelen ser más representativos que los más alejados. Este tipo de ponderación produce normalmente un suavizado más natural que una media uniforme. El kernel $[1, 4, 6, 4, 1] / 16$ constituye una aproximación discreta a un filtro gaussiano.

Cross-correlation

Are kernel sizes typically even or odd? Why?



Let F be the image, H be the kernel (of size $2k + 1 \times 2k + 1$), and G be the output image

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

This is called a cross-correlation

operation: $G = H \otimes F$

- Can think of as a “dot product” between local neighborhood and kernel for each pixel

22 of 119

Y ahora ya podemos introducir un concepto muy importante: la correlación cruzada. Si F representa la imagen de entrada, H es el kernel o filtro y G es la imagen de salida, la expresión matemática que aparece en la diapositiva define la operación de correlación cruzada. En esta asignatura utilizaremos para representarla el símbolo de un círculo con una cruz en su interior. Tened presente que esta notación no es universal y que distintos libros, cursos o librerías pueden utilizar símbolos diferentes.

Una forma intuitiva de entender la correlación cruzada es pensar en ella como un producto escalar entre el kernel y un vecindario local de la imagen. Colocamos el kernel sobre una determinada región de la imagen, multiplicamos cada valor del kernel por el píxel correspondiente y sumamos todos esos productos. El resultado es un único valor, que pasa a ocupar la posición correspondiente en la imagen de salida.

Esta operación se repite desplazando el kernel sobre la imagen. Por eso, para cada posición (i, j) , obtenemos un nuevo valor $G(i, j)$. Es decir, la correlación cruzada transforma la imagen de entrada en una nueva imagen cuyos valores dependen de cómo responde cada vecindario local al kernel utilizado.

Los índices de la expresión matemática pueden resultar algo confusos al principio, pero volveremos sobre ellos con un poco más de detalle posteriormente. Por ahora, lo importante es quedarse con la intuición: en cada posición calculamos el dot product (producto escalar) entre el kernel y el vecindario de la imagen situado bajo él.

Convolution

- Same as cross-correlation, except that the kernel is “flipped” (horizontally & vertically = bottom to top & right to left = rotate it 180°)

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i - u, j - v]$$

This is called a convolution operation:

$$G = H * F$$

- Convolution is **commutative** and **associative**

23 of 119

Ahora presentamos la convolución, que es una operación estrechamente relacionada con la correlación cruzada. El procedimiento es prácticamente el mismo, con una diferencia aparentemente pequeña pero importante: antes de aplicar el kernel, este se invierte en ambas dimensiones, lo que equivale a rotarlo 180 grados.

Este cambio se refleja directamente en la expresión matemática. En la correlación cruzada aparecía un término de la forma $F(i+u, j+v)$, mientras que en la convolución aparece como $F(i-u, j-v)$. En esta asignatura representaremos la convolución mediante un asterisco.

Podemos interpretar, por tanto, la convolución como una correlación cruzada realizada con el kernel previamente rotado 180 grados. Y este pequeño cambio introduce propiedades algebraicas muy útiles: bajo las condiciones habituales, la convolución es conmutativa,

$$f * h = h * f,$$

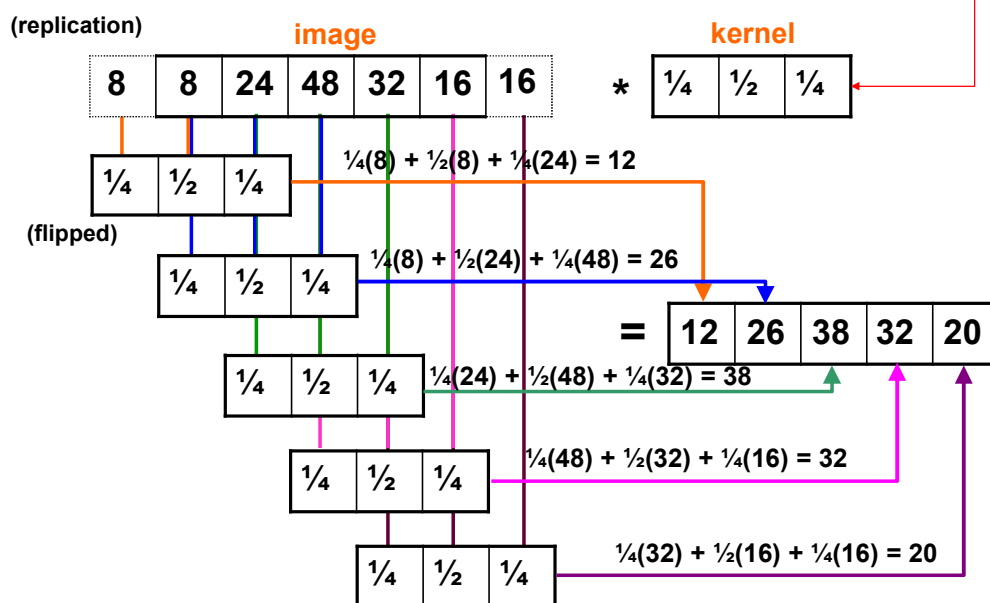
y también asociativa,

$$(f * h) * k = f * (h * k).$$

Estas propiedades no se cumplen, en general, para la correlación cruzada.

1D convolution example

What happens if you flip this kernel? 🤔



S. Birchfield, Clemson Univ., ECE 847, <http://www.ces.clemson.edu/~stb/ece847>

Independientemente de la notación matemática y de los índices, que al principio pueden resultar algo confusos, lo importante (tanto aquí como en el resto de la asignatura) es comprender la intuición subyacente y entender realmente cómo funciona la operación.

En esta diapositiva vemos un ejemplo unidimensional de convolución. Partimos de una señal 1D, que podemos interpretar como una imagen unidimensional, con los valores 8, 24, 48, 32 y 16. También tenemos un kernel 1D de tres elementos: $1/4$, $1/2$ y $1/4$.

Lo primero que deberíamos hacer para calcular una convolución es invertir/rotar el kernel. Sin embargo, en este caso el kernel es simétrico, por lo que al invertirlo obtenemos exactamente el mismo kernel. Por tanto, en este ejemplo concreto, convolución y correlación cruzada producen el mismo resultado.

También tenemos que decidir qué hacemos en los bordes, porque cuando situamos el kernel sobre el primer o el último elemento necesitamos valores que quedan fuera de la señal original. Existen distintas estrategias de padding, que veremos más adelante. En este ejemplo se extiende la señal utilizando nuevamente el valor del borde: colocamos un 8 a la izquierda y un 16 a la derecha.

A partir de ahí, simplemente vamos desplazando el kernel y calculando en cada posición el producto escalar entre el kernel y el vecindario correspondiente. Para el primer valor obtenemos:

$$1/4 \cdot 8 + 1/2 \cdot 8 + 1/4 \cdot 24 = 12.$$

Después desplazamos el kernel una posición hacia la derecha:

$$1/4 \cdot 8 + 1/2 \cdot 24 + 1/4 \cdot 48 = 26.$$

Y continuamos del mismo modo hasta recorrer toda la señal. De esta manera, la señal original

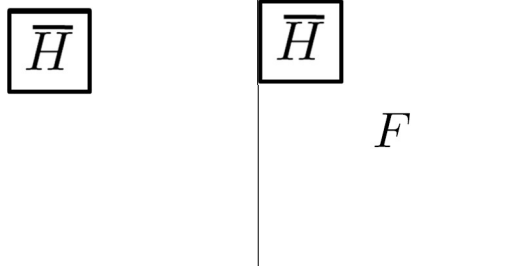
8, 24, 48, 32, 16

se transforma en

12, 26, 38, 32, 20.

Este ejemplo permite ver de forma muy sencilla qué hace realmente la convolución: desplazamos un kernel sobre la señal y, en cada posición, calculamos una suma ponderada de los valores del vecindario.

2D convolution



Adapted from F. Durand

25 of 119

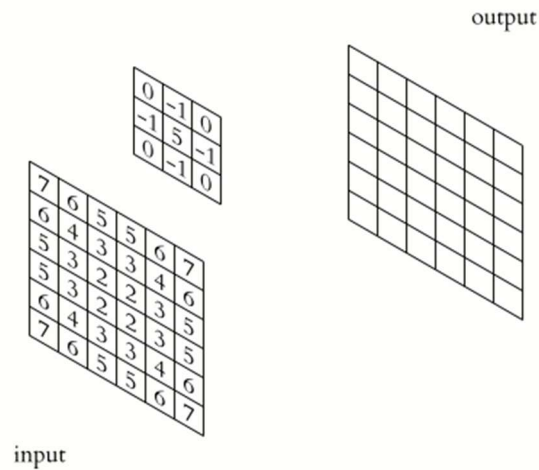
En esta diapositiva simplemente extendemos al caso bidimensional la misma idea que acabamos de ver en 1D.

Ahora tenemos una imagen F y un kernel bidimensional H . Para realizar la convolución, primero rotamos el kernel 180 grados, es decir, lo invertimos tanto horizontal como verticalmente.

A continuación, desplazamos ese kernel por las distintas posiciones de la imagen. En cada posición, superponemos el kernel sobre el vecindario correspondiente, multiplicamos elemento a elemento los valores del kernel y de la imagen, y sumamos todos esos productos. El resultado de esa operación se convierte en el nuevo valor del píxel correspondiente en la imagen de salida.

Por tanto, la idea es exactamente la misma que en 1D: desplazar el kernel, calcular una suma ponderada local y repetir la operación para todas las posiciones de la imagen.

2D convolution



https://upload.wikimedia.org/wikipedia/commons/1/19/2D_Convolution_Animation.gif

(using replicated boundaries)

En esta diapositiva reforzamos la misma idea mediante una animación. Partimos de una imagen de entrada y de un kernel 3×3 que se va desplazando por las distintas posiciones de la imagen.

En cada posición, el kernel se superpone sobre el vecindario correspondiente, se realiza la multiplicación elemento a elemento y se suman todos los productos para obtener el valor del píxel de salida.

En este ejemplo se utilizan bordes replicados, es decir, cuando el kernel necesita acceder a posiciones situadas fuera de la imagen, se reutiliza el valor del píxel más cercano del borde.

La animación permite visualizar de forma directa cómo la convolución va recorriendo toda la imagen y construyendo progresivamente la imagen de salida.

Cross-correlation vs Convolution

Cross-correlation

<i>Kernel H</i>				<i>Image F</i>		
1	2	3	$\otimes$	0	0	0
4	5	6		0	1	0
7	8	9		0	0	0

27 of 119

Ahora, en esta diapositiva y en las siguientes, vamos a desarrollar paso a paso un ejemplo muy sencillo que nos permitirá extraer algunas intuiciones importantes sobre la correlación cruzada y, posteriormente, sobre la convolución.

A la izquierda tenemos un kernel H de tamaño 3×3 . A la derecha tenemos la imagen F , formada por un único valor 1 en el centro y ceros en el resto de posiciones. Esta imagen es lo que se conoce como una señal impulso.

Lo que vamos a hacer primero es calcular la correlación cruzada entre F y H . En las siguientes diapositivas iremos desplazando el kernel sobre la imagen y viendo qué valores se obtienen en cada posición. Para obtener una imagen de salida del mismo tamaño que la imagen de entrada, necesitamos definir qué ocurre en los bordes, ya que al situar el kernel sobre los píxeles más cercanos al límite necesitamos acceder a posiciones que quedan fuera de la imagen. En este ejemplo vamos a utilizar zero padding, es decir, vamos a extender la imagen añadiendo ceros alrededor del borde. En este caso concreto, como todos los píxeles del borde de la imagen original ya son cero, utilizar zero padding produce exactamente el mismo resultado que replicar esos valores del borde.

Este ejemplo es especialmente útil porque una señal impulso permite visualizar de forma muy clara qué efecto tiene directamente el kernel sobre la imagen de salida.

Cross-correlation vs Convolution

Cross-correlation

$$\begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline 1 & 2 & 3 & \\ \hline 4 & 5 & 6 & 0 \\ \hline 7 & 8 & 9 & 0 \\ \hline & 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 9 & & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Cross-correlation

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 40 & 50 & 60 \\ \hline 70 & 81 & 90 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 9 & 8 & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Cross-correlation

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 0 & 4 & 0 & 5 & 0 & 6 \\ \hline 0 & 7 & 1 & 8 & 0 & 9 \\ \hline 0 & 0 & 0 & & & \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 9 & 8 & 7 \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Cross-correlation

Kernel H

1	2	3
4	5	6
7	8	9



Image F

0	0	0
0	1	0
0	0	0

=

1	2	3	0
4	5	6	0
7	8	9	0

=

9	8	7
6		

Cross-correlation vs Convolution

Cross-correlation

Kernel H

1	2	3
4	5	6
7	8	9



Image F

0	0	0
0	1	0
0	0	0

=

10	20	30
40	51	60
70	80	90

=

9	8	7
6	5	

Cross-correlation vs Convolution

Cross-correlation

Kernel H

1	2	3
4	5	6
7	8	9



Image F

0	0	0
0	1	0
0	0	0

=

0	10	20	3
0	41	50	6
0	70	80	9

=

9	8	7
6	5	4

Cross-correlation vs Convolution

Cross-correlation

Kernel H

1	2	3
4	5	6
7	8	9



Image F

0	0	0
0	1	0
0	0	0

=

		0	0	0
1	2	0	3	1
4	5	0	6	0
7	8	9		

=

9	8	7
6	5	4
3		

Cross-correlation vs Convolution

Cross-correlation

Kernel H

1	2	3
4	5	6
7	8	9



Image F

0	0	0
0	1	0
0	0	0

=

0	0	0
10	21	30
40	50	60
7	8	9

=

9	8	7
6	5	4
3	2	

Cross-correlation vs Convolution

Cross-correlation

Kernel H

1	2	3
4	5	6
7	8	9



Image F

0	0	0
0	1	0
0	0	0

=

0	0	0	
0	1	2	3
0	4	5	6
	7	8	9

=

9	8	7
6	5	4
3	2	1

Cross-correlation vs Convolution

Cross-correlation

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H & & \textit{Image } F \\ \hline 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ \hline \end{array}$$

37 of 119

Y ahora vamos a realizar exactamente la misma operación, pero intercambiando los papeles de la imagen y el kernel.

Es decir, lo que antes tratábamos como imagen ahora lo consideraremos como kernel, y viceversa. De nuevo, para poder mantener el mismo tamaño de salida y operar correctamente en los bordes, utilizaremos zero padding, añadiendo ceros alrededor de la imagen.

Este ejemplo nos permitirá comparar directamente qué ocurre al intercambiar ambos operandos y entender mejor una de las diferencias fundamentales entre correlación cruzada y convolución.

Cross-correlation vs Convolution

Cross-correlation

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline 0 & 0 & 0 & \\ \hline 0 & 1 & 1 & 0 & 2 & 3 \\ \hline 0 & 0 & 4 & 0 & 5 & 6 \\ \hline & 7 & 8 & 9 & & \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Cross-correlation

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 & 1 & 1 & 2 & 0 & 3 \\ \hline 0 & 4 & 0 & 5 & 0 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Cross-correlation

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline & 0 & 0 & 0 \\ \hline 1 & 0 & 2 & 1 & 3 & 0 \\ \hline 4 & 0 & 5 & 0 & 6 & 0 \\ \hline 7 & 8 & 9 & & & \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline & & & \\ \hline & & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Cross-correlation

Kernel H

0	0	0
0	1	0
0	0	0



Image F

1	2	3
4	5	6
7	8	9

=

0	01	02	3
0	14	05	6
0	07	08	9

=

1	2	3
4		

Cross-correlation vs Convolution

Cross-correlation

Kernel H

0	0	0
0	1	0
0	0	0



Image F

1	2	3
4	5	6
7	8	9

=

01	02	03
04	15	06
07	08	09

=

1	2	3
4	5	

Cross-correlation vs Convolution

Cross-correlation

Kernel H

0	0	0
0	1	0
0	0	0



Image F

1	2	3
4	5	6
7	8	9

=

1	02	03	0
4	05	16	0
7	08	09	0

=

1	2	3
4	5	6

Cross-correlation vs Convolution

Cross-correlation

Kernel H

0	0	0
0	1	0
0	0	0



Image F

1	2	3
4	5	6
7	8	9

=

	1	2	3
0	4	5	6
0	7	8	9
0	0	0	

=

1	2	3
4	5	6
7		

Cross-correlation vs Convolution

Cross-correlation

Kernel H

0	0	0
0	1	0
0	0	0



Image F

1	2	3
4	5	6
7	8	9

=

1	2	3
0 4	0 5	0 6
0 7	1 8	0 9
0	0	0

=

1	2	3
4	5	6
7	8	

Cross-correlation vs Convolution

Cross-correlation

Kernel H

0	0	0
0	1	0
0	0	0



Image F

1	2	3
4	5	6
7	8	9

=

1	2	3	
4	0	5	0
7	0	8	1
	0	0	0

=

1	2	3
4	5	6
7	8	9

Cross-correlation vs Convolution

Cross-correlation

$$\begin{array}{|c|c|c|} \hline \text{Kernel } H & & \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \text{Image } F & & \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 9 & 8 & 7 \\ \hline 6 & 5 & 4 \\ \hline 3 & 2 & 1 \\ \hline \end{array}$$

$$\begin{array}{|c|c|c|} \hline \text{Kernel } H & & \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \text{Image } F & & \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array}$$

Not commutative!!!!

47 of 119

Lo que vemos en este ejemplo es que no obtenemos el mismo resultado si intercambiamos el orden de los operandos en la correlación cruzada.

Es decir, calcular la correlación cruzada de la imagen con el kernel no produce, en general, el mismo resultado que calcular la correlación cruzada del kernel con la imagen.

Por tanto, este ejemplo actúa como un contraejemplo muy sencillo que nos permite concluir que la correlación cruzada no es una operación conmutativa.

Dicho de otra forma:

$$f \otimes h \neq h \otimes f$$

en general.

Cross-correlation vs Convolution

Convolution

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H & & \textit{Image } F \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array}$$

48 of 119

Ahora vamos a repetir exactamente el mismo ejercicio, pero utilizando la convolución en lugar de la correlación cruzada.

Mantendremos el mismo tratamiento de los bordes que en los ejemplos anteriores: utilizaremos zero padding, es decir, añadiremos ceros alrededor de la imagen para poder calcular la operación también en las posiciones próximas al borde y conservar el mismo tamaño de salida.

La diferencia fundamental respecto al caso anterior es que, al tratarse de una convolución, el kernel se invierte antes de aplicarlo.

Cross-correlation vs Convolution

Convolution

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline 9 & 8 & 7 & \\ \hline 6 & 5 & 4 & 0 \\ \hline 3 & 2 & 1 & 0 \\ \hline & 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Convolution

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 9 & 8 & 7 \\ \hline 60 & 50 & 40 \\ \hline 30 & 21 & 10 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

50 of 119

Cross-correlation vs Convolution

Convolution

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline & 9 & 8 & 7 \\ \hline 0 & 6 & 0 & 5 & 0 & 4 \\ \hline 0 & 3 & 1 & 2 & 0 & 1 \\ \hline 0 & 0 & 0 & & & \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

=

9	8	7	0
6	5	4	1
3	2	1	0

=

1	2	3
4		

Cross-correlation vs Convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

=

9	0	8	0	7	0
6	0	5	1	4	0
3	0	2	0	1	0

=

1	2	3
4	5	

53 of 119

Cross-correlation vs Convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

=

0	9	0	8	0	7
0	6	1	5	0	4
0	3	0	2	0	1

=

1	2	3
4	5	6

Cross-correlation vs Convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

=

		0	0	0
9	8	7	1	0
6	5	4	0	0
3	2	1		

=

1	2	3
4	5	6
7		

Cross-correlation vs Convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

=

0	0	0
9	0	8
6	0	5
3	2	1

=

1	2	3
4	5	6
7	8	

56 of 119

Cross-correlation vs Convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

=

0	0	0	
0	9	1	8
0	6	0	5
	3	2	1

=

1	2	3
4	5	6
7	8	9

Cross-correlation vs Convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

Cross-correlation vs Convolution

Convolution

$$\begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline 0 & 0 & 0 & \\ \hline 0 & 1 & 1 & 0 & 2 & 3 \\ \hline 0 & 0 & 4 & 0 & 5 & 6 \\ \hline & 7 & 8 & 9 & & \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Convolution

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 & 1 & 1 & 2 & 0 & 3 \\ \hline 0 & 4 & 0 & 5 & 0 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Convolution

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \textit{Image } F \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline & 0 & 0 & 0 \\ \hline 1 & 0 & 2 & 1 & 3 & 0 \\ \hline 4 & 0 & 5 & 0 & 6 & 0 \\ \hline 7 & 8 & 9 & & & \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

Cross-correlation vs Convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

0	0	1	0	2	3
0	1	4	0	5	6
0	0	7	0	8	9

=

1	2	3
4		

Cross-correlation vs Convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

01	02	03
04	15	06
07	08	09

=

1	2	3
4	5	

Cross-correlation vs Convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

1	02	03	0
4	05	16	0
7	08	09	0

=

1	2	3
4	5	6

Cross-correlation vs Convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

		1	2	3
0	4	5	6	
0	7	8	9	
0	0	0		

=

1	2	3
4	5	6
7		

Cross-correlation vs Convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

1	2	3
0 4	0 5	0 6
0 7	1 8	0 9
0	0	0

=

1	2	3
4	5	6
7	8	

Cross-correlation vs Convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

1	2	3	
4	0	5	0
7	0	8	1
	0	0	0

=

1	2	3
4	5	6
7	8	9

Cross-correlation vs Convolution

Convolution

$$\begin{array}{|c|c|c|} \hline \text{Kernel } H & & \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \text{Image } F & & \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array}$$

$$\begin{array}{|c|c|c|} \hline \text{Kernel } H & & \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \text{Image } F & & \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array}$$

Commutative!!!!

68 of 119

Hemos repetido el experimento intercambiando los dos operandos y hemos comprobado que, en el caso de la convolución, el resultado sí es el mismo independientemente del orden. Es decir, se cumple la propiedad conmutativa:

$$f \star h = h \star f.$$

Esta es una de las propiedades fundamentales de la convolución. La diferencia respecto a la correlación cruzada está precisamente en la inversión espacial que forma parte de la definición de la convolución: el kernel se rota 180 grados antes de realizar la suma ponderada.

La inversión del kernel que introduce la definición de convolución es precisamente lo que permite que esta operación, a diferencia de la correlación cruzada, sea conmutativa.

Cross-correlation vs Convolution

Cross-correlation

$$\begin{array}{|c|c|c|} \hline \text{Kernel } H \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \text{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 9 & 8 & 7 \\ \hline 6 & 5 & 4 \\ \hline 3 & 2 & 1 \\ \hline \end{array}$$

$5 \cdot 0 + 6 \cdot 0 + 8 \cdot 0 + 9 \cdot 1$ $4 \cdot 0 + 5 \cdot 0 + 6 \cdot 0 + 7 \cdot 0 + 8 \cdot 1 + 9 \cdot 0$

If the input is an impulse signal, cross-correlation produces the filter rotated 180°. **Convolution gives a more natural output:** if the mask is convolved with an impulse signal, it provides exactly the same mask as output.

Convolution

$$\begin{array}{|c|c|c|} \hline \text{Kernel } H \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \text{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline \text{Kernel } H \\ \text{(rotated } 180^\circ) \\ \hline 9 & 8 & 7 \\ \hline 6 & 5 & 4 \\ \hline 3 & 2 & 1 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline \text{Image } F \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array}$$

$5 \cdot 0 + 4 \cdot 0 + 2 \cdot 0 + 1 \cdot 1$ $6 \cdot 0 + 5 \cdot 0 + 4 \cdot 0 + 3 \cdot 0 + 2 \cdot 1 + 1 \cdot 0$

69 of 119

Si continuamos razonando sobre el resultado que acabamos de observar, podemos extraer una intuición muy útil sobre la diferencia entre correlación cruzada y convolución.

Si la entrada es una señal impulso, es decir, una imagen que vale 1 únicamente en una posición y 0 en todas las demás, la correlación cruzada produce como salida el filtro rotado 180 grados. Esto es consecuencia directa de la forma en que está definida la operación.

En cambio, con la convolución obtenemos un resultado, podríamos decir, más intuitivo y natural: al convolucionar una señal impulso con un kernel, la salida es exactamente el propio kernel. Dicho de otra manera, el impulso actúa como el elemento identidad de la convolución.

Por tanto:

$\text{impulso} \star h = h.$

Cross-correlation vs Convolution (technical note)

- Functions like `cv2.filter2D` actually apply cross-correlation and not convolution!!!
- Other functions, like `scipy.signal.convolve2d`, do flip the kernel.

<code>cv2.filter2D(im, -1, cv2.flip(fil, -1))</code>	} To obtain exactly the same result with both functions, we also need to use compatible boundary conditions.
equivalent to	
<code>signal.convolve2d(im, fil, mode='same')</code>	
- So, please, check the documentation before using the functions!!!

70 of 119

Y en este punto conviene hacer una anotación técnica importante. Aunque la convolución tiene mucho interés tanto desde el punto de vista matemático como conceptual, algunas funciones muy utilizadas no implementan literalmente una convolución.

Por ejemplo, `cv2.filter2D` de OpenCV aplica en realidad una correlación cruzada: utiliza el kernel tal y como se le proporciona, sin rotarlo 180 grados. Esto no supone ningún problema; simplemente tenemos que saber qué operación está realizando la función para utilizarla correctamente.

Si queremos implementar una convolución con `filter2D`, basta con proporcionar previamente el kernel rotado 180 grados. De esta forma, la correlación cruzada que realiza internamente OpenCV produce el resultado de una convolución.

En cambio, otras funciones, como `scipy.signal.convolve2d`, sí implementan directamente la convolución y realizan internamente la inversión del kernel.

Por tanto, la idea importante no es que una biblioteca lo haga "bien" y otra "mal", sino que debemos conocer la definición concreta que implementa cada función. Por eso es importante consultar siempre la documentación y saber si una determinada operación utiliza el kernel tal cual o lo invierte antes de aplicarlo.

Cross-correlation vs Convolution (technical note)

```
import scipy.signal as sp
import numpy as np
import cv2

image = np.array([[0, 0, 0], [0,1,0],[0,0,0]], dtype=np.uint8)
filter = np.array([[1, 2, 3], [4,5,6],[7,8,9]], dtype=np.uint8)

print('Result using sp.convolve2d')
print(sp.convolve2d(image, filter, mode='same'))

print('Result using cv2.filter2D')
print(cv2.filter2D(image, -1, filter, borderType=cv2.BORDER_CONSTANT))

print('Result using cv2.filter2D and cv2.flip')
print(cv2.filter2D(image, -1, cv2.flip(filter,-1), borderType=cv2.BORDER_CONSTANT))
```

```
Result using sp.convolve2d
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Result using cv2.filter2D
[[9 8 7]
 [6 5 4]
 [3 2 1]]
Result using cv2.filter2D and cv2.flip
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

71 of 119

En esta diapositiva tenéis también una nota técnica en la que os muestro un código completamente funcional. Es decir, si copiáis este fragmento y lo pegáis en una celda de Google Colab, podréis ejecutarlo directamente y obtener el resultado que se muestra a la derecha.

La idea es que utilizéis este pequeño ejemplo también para experimentar. Podéis modificar los distintos parámetros de las funciones y observar cómo cambia el resultado. Esto nos ayuda a familiarizarnos con el comportamiento real de las funciones y con su documentación.

En particular, el ejemplo permite comparar `scipy.signal.convolve2d`, que implementa directamente la convolución, con `cv2.filter2D`, que realiza correlación cruzada. Con `filter2D` podemos probar tanto el kernel original como el kernel rotado 180 grados y comprobar de forma práctica la diferencia entre ambas operaciones.

En **negrita** destacamos dos elementos que explicaremos a continuación: el modo en que se realiza la convolución y el tipo de borde.

Cross-correlation vs Convolution (technical note)

```
import scipy.signal as sp
import cv2
```

```
sp.convolve2d()
https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.convolve2d.html
```

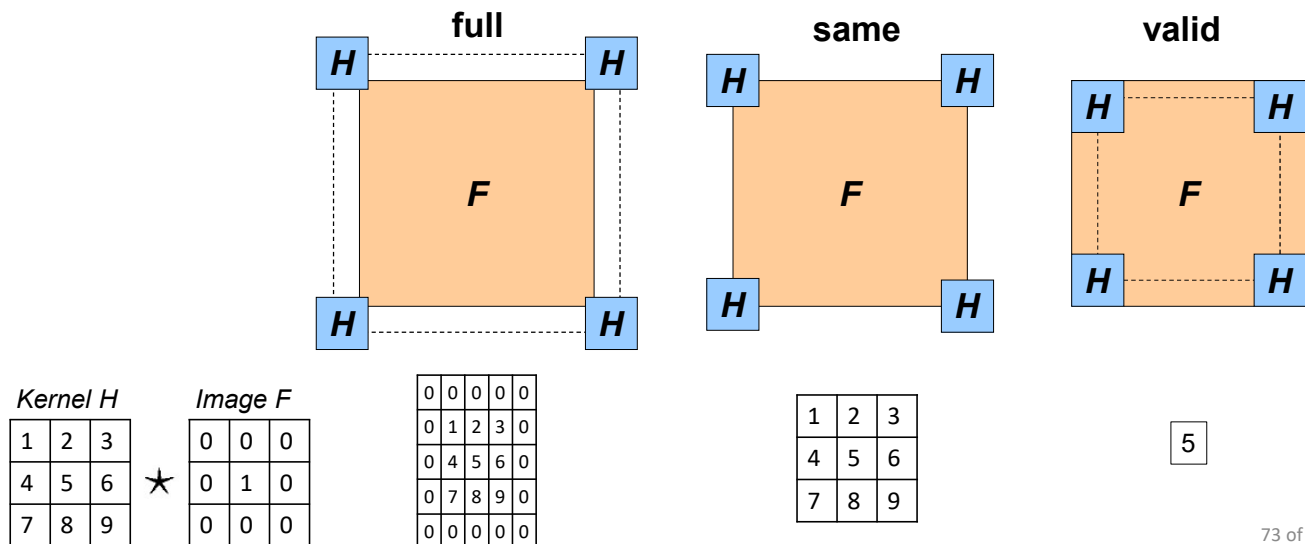
```
sp.correlate2d()
https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.correlate2d.html
```

```
cv2.filter2D()
https://docs.opencv.org/4.x/d4/d86/group\_imgproc\_filter.html#ga27c049795ce870216ddfb366086b5a04
```

72 of 119

Boundary issues

- What is the size of the output?
- Output size / “shape” options



73 of 119

Ahora vamos a analizar qué ocurre en los bordes y cómo esto afecta al tamaño de la salida de una correlación o una convolución. Hay tres modos clásicos: full, same y valid. Esto enlaza directamente con la nota técnica que vimos anteriormente, donde aparecían en negrita ciertos parámetros de las funciones relacionados con el modo de la operación y con el tratamiento de los bordes.

En el modo full, permitimos todos los solapamientos posibles entre la imagen F y el kernel H, incluso aquellos en los que únicamente un elemento del kernel coincide con un píxel de la imagen. Podemos imaginar que comenzamos colocando el kernel en la posición más extrema posible y lo desplazamos hasta el extremo opuesto. Por ello, la imagen de salida es mayor que la imagen original. Si tanto la imagen como el kernel son de 3×3, obtenemos una salida de 5×5.

En el modo same, como indica su nombre, nos quedamos con una salida del mismo tamaño que la imagen de entrada (dependiendo de la implementación, si el kernel es más grande que la señal, la salida será del tamaño del kernel; o, en otras ocasiones, se define same como "mismo tamaño que el primer argumento"). Así, si tenemos una imagen de 3×3 y un kernel de 3×3, la salida será también de 3×3.

Para calcular los valores próximos a los bordes necesitamos definir de alguna manera qué ocurre fuera de la imagen.

Finalmente, en el modo valid solo se consideran las posiciones en las que existe un solapamiento completo entre ambos operandos. Por tanto, no necesitamos utilizar ningún tipo de padding y la salida es más pequeña. En nuestro ejemplo, con una imagen de 3×3 y un kernel también de 3×3, solamente existe una posición en la

que el kernel cabe completamente dentro de la imagen, de modo que la salida consta de un único valor.

Boundary issues

- What about near the edge?
 - the filter window falls off the edge of the image
 - need to extrapolate
 - methods:
 - clip filter (black) (a.k.a. zero padding)
 - wrap around
 - copy edge
 - reflect across edge



Source: S. Marschner 74 of 119

Si vamos a trabajar en modo full o same, aparece una cuestión adicional: qué valores suponemos que existen fuera de los límites de la imagen. En modo valid este problema no aparece, porque únicamente se calculan aquellas posiciones en las que existe un solapamiento completo entre ambos operandos.

Para poder operar cerca de los bordes necesitamos, por tanto, extender de alguna manera la imagen. Existen distintas estrategias y la elección dependerá de la aplicación concreta. La más sencilla y probablemente la más habitual es el zero padding, que consiste en asumir que todos los valores situados fuera de la imagen son cero. Esto encaja además de forma natural con la definición matemática habitual de convolución de señales de soporte finito: podemos considerar que tanto la señal como el filtro valen cero fuera de su dominio o soporte conocido.

Pero no es la única posibilidad. Podemos replicar el borde, utilizando fuera de la imagen el mismo valor del píxel más cercano; podemos reflejar la imagen, generando una especie de espejo en el borde; o podemos utilizar un wrap-around o extensión periódica, en la que los valores que aparecen por un lado de la imagen proceden del lado opuesto.

Por tanto, no existe una estrategia universalmente mejor. El método de extensión debe elegirse teniendo en cuenta la naturaleza de la imagen, la operación que estamos realizando y la aplicación concreta, ya que diferentes tratamientos de los bordes pueden producir resultados distintos,

especialmente en las proximidades de estos.

Boundary issues

- What about near the edge?
 - the filter window falls off the edge of the image
 - need to extrapolate
 - Types (OpenCV):

```
* BORDER_REPLICATE: aaaaaa | abcdefgh | hhhhhhh  
* BORDER_REFLECT: fedcba | abcdefgh | hgfedcb  
* BORDER_REFLECT_101: gfedcb | abcdefgh | gfedcba ← Default value  
* BORDER_WRAP: cdefgh | abcdefgh | abcdefg  
* BORDER_CONSTANT: iiiiii | abcdefgh | iiiiii
```

75 of 119

Estas distintas estrategias de tratamiento de los bordes están ya implementadas, por ejemplo, en OpenCV mediante los diferentes valores de `borderType`. En esta diapositiva utilizamos como ejemplo una señal unidimensional formada por los valores A, B, C, D, E, F, G y H.

Con `BORDER_REPLICATE`, simplemente repetimos el valor situado en el extremo tantas veces como sea necesario. Por tanto, hacia la izquierda de A tendremos más valores A y, hacia la derecha de H, más valores H.

Con `BORDER_REFLECT`, reflejamos la señal como si colocásemos un espejo en cada extremo, incluyendo el propio valor del borde. Así, inmediatamente a la izquierda de A aparece otro A, después B, C, etc.; y a la derecha de H aparece otro H, después G, F, etc.

`BORDER_REFLECT_101`, que es el utilizado por defecto en OpenCV, hace algo muy parecido, pero sin repetir el píxel situado exactamente en el borde. Por ejemplo, a la izquierda de A aparecería primero B, después C, etc.; y a la derecha de H aparecería primero G, después F, etc.

Con `BORDER_WRAP` realizamos una extensión periódica: al llegar a un extremo continuamos con los valores procedentes del extremo contrario. Por tanto, inmediatamente a la izquierda de A aparecería H, y a la derecha de H aparecería A.

Finalmente, con `BORDER_CONSTANT` rellenamos todas las posiciones exteriores con un determinado valor constante, representado en esta

diapositiva mediante la letra I.

Cross-correlation vs Convolution

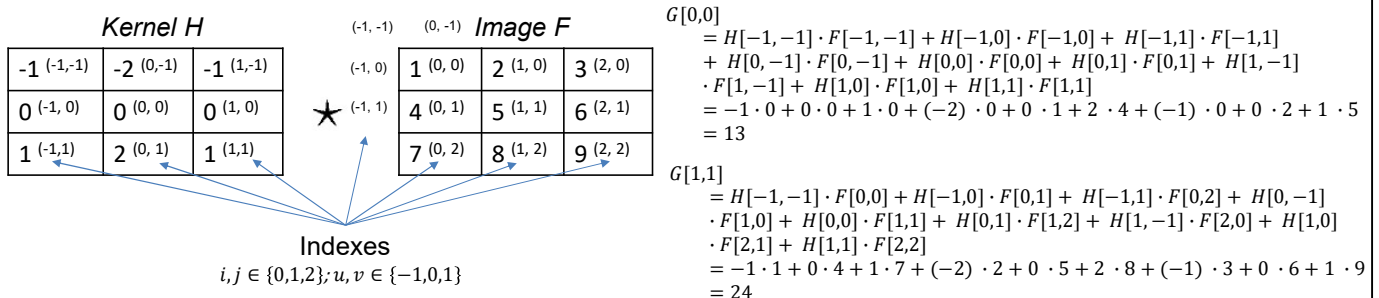
Cross-correlation

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

Convolution

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i - u, j - v]$$

How exactly do these indices work? Let's see an example using cross-correlation (convolution is the same: just flip the filter in both dimensions, then apply cross-correlation)



Note: The important thing is that you perform the dot product of an image and a filter (in the case of convolution, rotated 180°), and replace the central value.

Nice reference on convolution:

<https://www.songho.ca/dsp/convolution/convolution.html>

En esta diapositiva bajamos un poco más al detalle de los índices, no porque sea necesario memorizar esta expresión, sino para entender exactamente qué significa la operación. En la imagen F, los índices (i,j) identifican la posición en la que queremos calcular el píxel de salida, mientras que (u,v) representan desplazamientos dentro del kernel H, cuyo centro consideramos la posición (0,0).

En el caso de la correlación cruzada, para calcular $G[i,j]$ colocamos el centro del kernel sobre la posición (i,j) de la imagen (cuyo origen, como ya hemos visto, se encuentra en la esquina superior izquierda). A continuación, cada elemento $H[u,v]$ se multiplica por el píxel correspondiente $F[i+u,j+v]$, y finalmente sumamos todos esos productos. Es decir, estamos calculando simplemente un producto escalar entre el kernel y el vecindario de la imagen.

En la diapositiva se muestran dos ejemplos. Para calcular $G[0,0]$, el kernel está centrado sobre la esquina superior izquierda de la imagen, por lo que algunas de las posiciones necesarias quedan fuera de ella. En este ejemplo se supone que esas posiciones valen cero (zero padding). Al realizar todos los productos y sumarlos obtenemos 13. En cambio, al calcular $G[1,1]$, el kernel queda completamente contenido dentro de la imagen y el resultado es 24.

La convolución funciona exactamente con la misma mecánica, con una única diferencia fundamental: antes de realizar esta operación rotamos el kernel 180 grados. Una vez rotado, podemos aplicar exactamente el mismo procedimiento que acabamos de describir para la correlación cruzada.

La idea importante, por tanto, no es quedarse con los índices concretos, sino con la intuición: centramos el kernel en una posición, multiplicamos elemento a elemento con el vecindario correspondiente, sumamos los productos y obtenemos un nuevo píxel de salida.

Key properties of convolution

Linearity: if f and g are images, a and b scalars, and L a linear filter/operator:

$$L(af+bg) = aL(f) + bL(g)$$

homogeneity (or scaling): $L(af) = aL(f)$
+
additivity: $L(f+g) = L(f) + L(g)$

Shift equivariance: same behavior regardless of pixel location. Operator behaves the same everywhere, i.e. the value of the output depends on the pattern in the image neighborhood, not the position of the neighborhood.

$$L(\text{shift}(f)) = \text{shift}(L(f))$$

In the case of shift invariance we'd have: $L(\text{shift}(f)) = L(f)$

77 of 119

En esta diapositiva resumimos dos propiedades fundamentales de la convolución: linealidad y equivarianza frente a traslaciones.

En primer lugar, la convolución es una operación lineal. Esto significa que si tenemos dos imágenes f y g , y dos escalares a y b , aplicar el operador lineal L a la combinación $af+bg$ produce exactamente el mismo resultado que aplicar el operador por separado a cada imagen, multiplicar después cada resultado por el escalar correspondiente y sumarlos:

$$L(af+bg) = aL(f) + bL(g)$$

Esta propiedad combina dos ideas: homogeneidad, según la cual multiplicar la entrada por un escalar multiplica la salida por ese mismo escalar, y aditividad, según la cual filtrar la suma de dos imágenes equivale a sumar las imágenes filtradas por separado. Juntas constituyen el principio de superposición.

La segunda propiedad es la equivarianza frente a traslaciones. La idea intuitiva es que el filtro se comporta de la misma manera independientemente de dónde aparezca un determinado patrón en la imagen. Si desplazamos la imagen de entrada y después aplicamos la convolución, obtenemos el mismo resultado que si primero aplicamos la convolución y después desplazamos la salida:

$$L(\text{shift}(f)) = \text{shift}(L(f))$$

Por tanto, la respuesta depende del patrón local que encuentra el filtro, no de la posición absoluta en la que ese patrón aparece.

Es importante distinguir equivarianza de invariancia. Equivarianza significa que, si desplazamos la entrada, la salida se desplaza de la misma manera. Invariancia sería algo diferente: que la salida no cambiase al desplazar la entrada, es decir,

$$L(\text{shift}(f)) = L(f)$$

La convolución es, en general, equivariante a traslaciones, no invariante a traslaciones.

More properties of convolution

- Commutative: $a * b = b * a$
 - Conceptually no difference between filter and signal (image)
- Associative: $a * (b * c) = (a * b) * c$
 - Often apply several filters one after another: $((a * b_1) * b_2) * b_3$
 - This is equivalent to applying one filter: $a * (b_1 * b_2 * b_3)$
- Distributes over addition: $a * (b + c) = (a * b) + (a * c)$
- Scalars factor out: $ka * b = a * kb = k(a * b)$
- Identity: unit impulse $e = [0, 0, 1, 0, 0]$,
 $a * e = a$

Source: S. Lazebnik

78 of 119

En esta diapositiva recopilamos algunas propiedades algebraicas adicionales de la convolución que resultan especialmente útiles tanto desde el punto de vista matemático como computacional.

En primer lugar, la convolución es conmutativa: $a * b = b * a$.

Matemáticamente, por tanto, podemos intercambiar los dos operandos sin modificar el resultado. Esto conecta con lo que vimos anteriormente: si convolucionamos una imagen con un filtro obtenemos el mismo resultado que si convolucionamos el filtro con la imagen.

Aunque en una aplicación concreta solemos distinguir conceptualmente entre “imagen” y “kernel”, desde el punto de vista de la operación de convolución ambos son simplemente señales.

La convolución también es asociativa: $a * (b * c) = (a * b) * c$. Esta propiedad es especialmente interesante cuando aplicamos varios filtros consecutivamente. En lugar de convolucionar primero la imagen con b_1 , después el resultado con b_2 y finalmente con b_3 , sabemos que eso es equivalente a convolucionar previamente los filtros entre sí y aplicar a la imagen un único filtro. Es decir, $((a * b_1) * b_2) * b_3$ es equivalente a $a * (b_1 * b_2 * b_3)$.

También es distributiva respecto de la suma: $a * (b + c) = (a * b) + (a * c)$

c). Por tanto, convolucionar una señal con la suma de dos filtros equivale a convolucionarla por separado con cada uno de ellos y sumar posteriormente los resultados.

Los escalares también pueden factorizarse. Multiplicar uno de los operandos por una constante k equivale a multiplicar por k el resultado final de la convolución: $(k a) * b = a * (k b) = k (a * b)$.

Finalmente, la convolución tiene un elemento identidad: el impulso unidad. Si convolucionamos cualquier señal a con un impulso, obtenemos exactamente la propia señal a . Esta es precisamente la propiedad que vimos anteriormente cuando comprobamos que convolucionar un kernel con una señal impulso nos devuelve el propio kernel.

Como matiz técnico, estas propiedades son exactas para la convolución matemática. Cuando trabajamos con imágenes finitas y funciones concretas de software, el tratamiento de los bordes y el modo de salida utilizado, por ejemplo, full, same o valid, pueden afectar a cómo se manifiestan estas propiedades en la práctica.

Cross-correlation vs Convolution

Cross-correlation	Convolution
Measurement of similarity between two signals/sequences. Commonly used for template matching.	Linear filtering: computes the effect of a filter on a signal.
If the filter is symmetric under a 180° rotation , like a Gaussian or a Laplacian, it makes no difference . When the filter is not symmetric, like a derivative, it makes a difference.	
Not associative.	Associative
Not commutative.	Commutative.
Useful when the goal is to compare an image with a template or pattern . No kernel flipping is required.	Associativity means that a sequence of convolutional filters can be represented by a single equivalent filter: $(f * g) * h = f * (g * h)$ Associativity is especially useful for understanding and designing combinations of filters and for choosing alternative implementations. Computational savings arise only in particular cases, for example when additional structure such as separability can be exploited.

79 of 119

En esta diapositiva resumimos algunas de las principales diferencias entre correlación cruzada y convolución.

La correlación cruzada se utiliza habitualmente para medir el grado de similitud entre dos señales o entre una imagen y un patrón. Es precisamente la idea que aparece, por ejemplo, en template matching: desplazamos una plantilla sobre la imagen y buscamos aquellas posiciones en las que existe una mayor correspondencia entre ambas.

La convolución, en cambio, se interpreta habitualmente como una operación de filtrado lineal: una señal, que en nuestro caso será una imagen, es transformada mediante la aplicación de un determinado filtro.

Hay un caso importante en el que ambas operaciones producen exactamente el mismo resultado. Si el kernel permanece idéntico después de rotarlo 180 grados, correlación cruzada y convolución coinciden. Esto ocurre, por ejemplo, con muchos filtros gaussianos o laplacianos. En cambio, con kernels que no presentan esta simetría, como determinados filtros de derivación, las dos operaciones producen resultados diferentes.

Otra diferencia fundamental es que la correlación cruzada, en general, no es ni conmutativa ni asociativa. La convolución, por el contrario, sí posee ambas propiedades.

La asociatividad significa que, si aplicamos sucesivamente dos filtros g y h a una imagen f , podemos reagrupar las operaciones sin modificar el resultado

matemático:

$$(f * g) * h = f * (g * h)$$

Por tanto, una sucesión de filtros convolucionales puede representarse mediante un único filtro equivalente obtenido al convolucionar los filtros originales. Esto es conceptualmente muy útil porque nos permite entender una cadena de filtros como un único sistema convolucional y también nos permite escoger distintas formas de implementar una misma operación.

Es importante destacar, sin embargo, que disponer de un único kernel equivalente no implica necesariamente una ventaja computacional. Por ejemplo, aplicar sucesivamente dos kernels de 3x3 requiere 18 multiplicaciones por cada píxel calculado, mientras que el kernel equivalente obtenido mediante convolución full tendría un tamaño de 5x5 y requeriría 25 multiplicaciones por píxel si se aplicase directamente. La asociatividad garantiza equivalencia matemática, pero no necesariamente una reducción del número de operaciones. Pre-convolucionar o fusionar kernels no reduce el número de FLOPs. Sin embargo, puede aportar ventajas prácticas al reducir el número de pasadas sobre la imagen y los accesos a memoria.

Las ventajas computacionales más claras aparecen cuando podemos explotar alguna estructura adicional de los filtros. Un ejemplo especialmente importante es la separabilidad, que veremos más adelante: determinados filtros bidimensionales pueden descomponerse exactamente en filtros unidimensionales sucesivos, reduciendo considerablemente el coste computacional.

Por tanto, para template matching o para comparar directamente una imagen con un patrón, la correlación cruzada resulta una operación natural y no requiere invertir el kernel. La convolución, por su parte, posee propiedades como la conmutatividad y la asociatividad, que resultan especialmente útiles para analizar y combinar sistemas de filtrado.

Linear filters: examples



Image



0	0	0
0	1	0
0	0	0



?

mode='same' con zero padding

Source: D. Lowe

80 of 119

¿Cuál sería el resultado de esta convolución?

Linear filters: examples



Image



0	0	0
0	1	0
0	0	0



Identical image

mode='same' with zero padding

Source: D. Lowe

81 of 119

Linear filters: examples



Image

$$\begin{matrix} * & \begin{matrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{matrix} & = & ? \end{matrix}$$

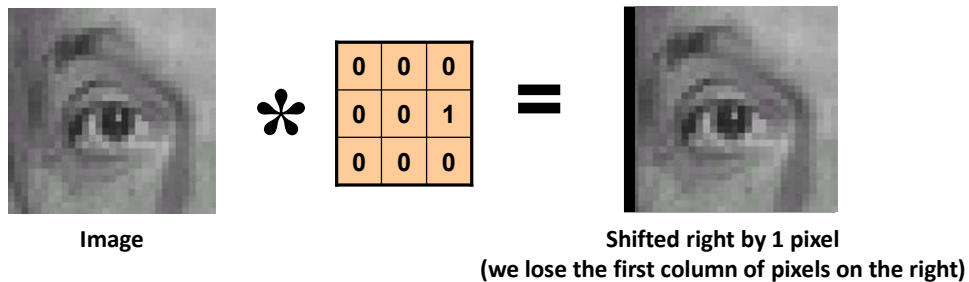
mode='same' con zero padding

Source: D. Lowe

82 of 119

¿Cuál sería el resultado de esta convolución?

Linear filters: examples



mode='same' con zero padding

Source: D. Lowe

83 of 119

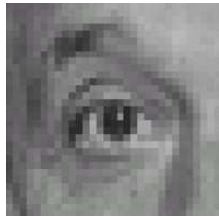
Al realizar la convolución, rotamos el kernel 180 grados. En este caso, después de la rotación, la primera y la última fila del kernel 3x3 contienen únicamente ceros, mientras que la fila central es 1, 0, 0. Como el valor que calculamos se asigna a la posición correspondiente al centro del kernel, en cada posición estamos tomando el valor del píxel situado inmediatamente a la izquierda y copiándolo en la posición actual.

Por tanto, cada píxel de la imagen original aparece una posición más a la derecha en la imagen de salida. Ésa es la razón por la que observamos que la imagen del ojo se desplaza hacia la derecha.

Además, como estamos utilizando zero padding, cuando calculamos la primera columna de la salida, el píxel que necesitamos consultar se encuentra fuera de la imagen y su valor se considera cero. Por eso toda la columna situada más a la izquierda queda a cero. Al mismo tiempo, la última columna de la imagen original se desplazaría fuera del dominio de la imagen de salida y, al mantener el mismo tamaño, se pierde.

La intuición esencial es, por tanto, muy sencilla: con este kernel, cada posición de salida copia el valor de su vecino izquierdo, lo que equivale a desplazar toda la imagen una posición hacia la derecha.

What if this was a correlation operation???



Image



0	0	0
0	0	1
0	0	0

= ?

mode='same' con zero padding

84 of 119

¿Qué pasaría ahora si, en lugar de convolución, aplicásemos una correlación?

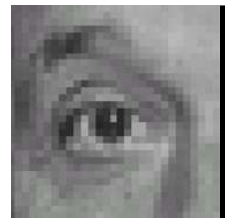
What if this was a correlation operation???



Image



0	0	0
0	0	1
0	0	0



Shifted left by 1 pixel
(we lose the first column of pixels on the left)

mode='same' con zero padding

85 of 119

En el caso de la correlación cruzada, la lógica es exactamente la misma, pero ahora el kernel se utiliza tal y como está, sin rotarlo 180 grados.

Por tanto, si el kernel original tiene en su fila central los valores 0, 0, 1, cada píxel de salida toma el valor del píxel situado inmediatamente a su derecha. Como consecuencia, cada píxel de la imagen original aparece una posición más a la izquierda en la imagen de salida y, por tanto, la imagen del ojo se desplaza hacia la izquierda.

Linear filters: examples

Image **Kernel**

0	0	0
0	0	1
0	0	0

 $\otimes$  $= ?$

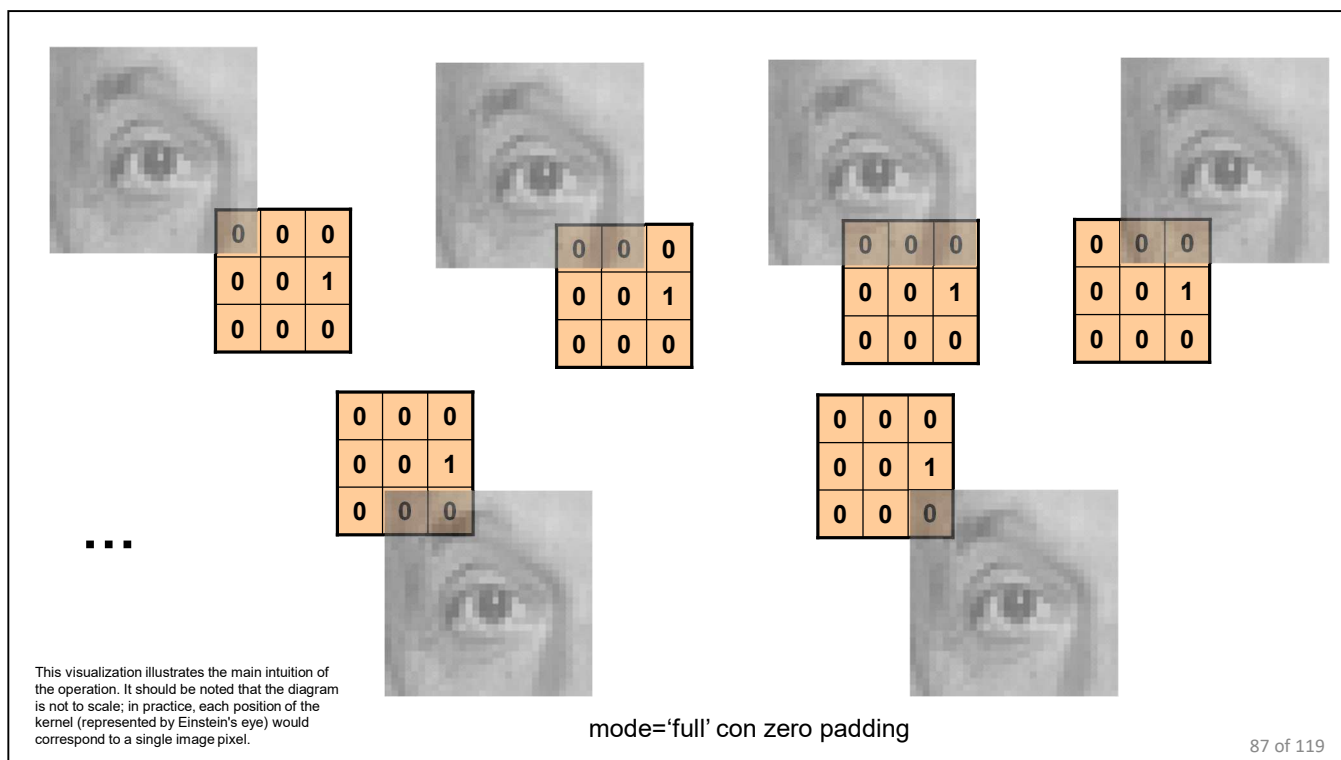
mode='full' con zero padding

86 of 119

Ahora jugamos un poco más con la operación. En este ejemplo tenemos una imagen muy pequeña, de 3x3, que vamos a correlacionar con un kernel mucho más grande, que en este caso es la imagen del ojo.

La pregunta es simplemente: ¿qué resultado obtendríamos al realizar esta correlación? Este ejemplo tiene sobre todo un propósito didáctico, para ayudarnos a pensar qué significa realmente desplazar un kernel sobre una imagen y cómo influyen los tamaños relativos de ambos operandos.

En aplicaciones habituales de filtrado, lo normal es que ocurra justamente lo contrario: la imagen es mucho más grande y el kernel tiene un tamaño relativamente pequeño, por ejemplo 3x3, 5x5 o 7x7. Aquí hemos invertido deliberadamente esa situación para jugar con la definición de la correlación y reforzar la intuición sobre la operación.



87 of 119

En esta diapositiva vemos paso a paso qué ocurre cuando realizamos la correlación cruzada entre una imagen muy pequeña de 3x3, que contiene un único 1 y ceros en el resto de posiciones, y un kernel mucho más grande, que en este caso es la imagen del ojo.

Estamos trabajando en modo full y utilizando zero padding, por lo que consideramos todos los posibles solapamientos entre la imagen y el kernel, incluso aquellos en los que únicamente una pequeña parte de ambos coincide.

La intuición es especialmente sencilla porque la imagen 3x3 contiene un único valor distinto de cero. En cada posición, todos los productos son cero excepto aquel en el que el 1 coincide con un determinado píxel del kernel. Por tanto, cada valor de la salida acaba siendo simplemente uno de los valores de la imagen del ojo.

A medida que desplazamos el kernel y consideramos todos los posibles solapamientos, ese 1 va seleccionando sucesivamente distintos píxeles del kernel. Como estamos utilizando correlación cruzada y no rotamos previamente el kernel, el resultado asociado a una señal impulso corresponde al kernel rotado 180 grados, además del desplazamiento determinado por la posición concreta del impulso dentro de la imagen 3x3.

Las franjas negras que aparecen en la salida son consecuencia directa

del modo full y de que la imagen 3x3 contiene ceros alrededor del 1. Existen posiciones extremas en las que los únicos valores de la imagen pequeña que se solapan con el kernel son precisamente esos ceros, por lo que el resultado en esas posiciones también es cero.

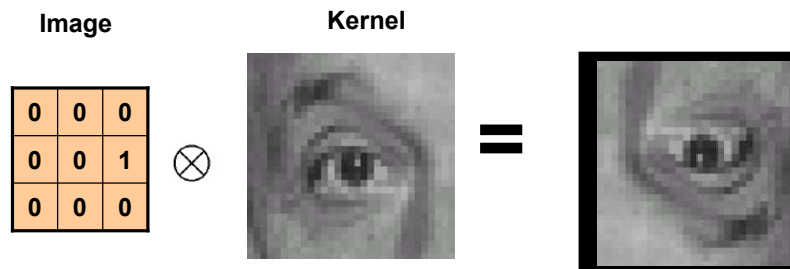
Verticalmente, como el 1 está situado en la fila central de la imagen 3x3, aparece una fila de ceros por encima y otra por debajo del contenido procedente del kernel.

Horizontalmente, como el 1 está situado en la columna derecha de la imagen 3x3, la respuesta queda desplazada hacia la derecha dentro de la salida full, apareciendo columnas de ceros en el lado izquierdo. De hecho, con un impulso situado en la tercera columna de una imagen 3x3 deberían aparecer dos columnas de ceros a la izquierda si se representa la correlación full completa.

La idea general es que, en modo full, el tamaño de la salida aumenta. Si una de las entradas mide 3x3, la salida tendrá dos posiciones adicionales en cada dimensión respecto al tamaño de la otra entrada. La posición concreta del impulso dentro de ese 3x3 determina cómo se reparten esas posiciones adicionales entre la parte superior e inferior y entre la izquierda y la derecha.

Este ejemplo es deliberadamente poco habitual desde el punto de vista práctico, porque normalmente el kernel es mucho más pequeño que la imagen. Su interés es puramente didáctico: permite visualizar con claridad cómo se construye la salida a partir de todos los posibles solapamientos.

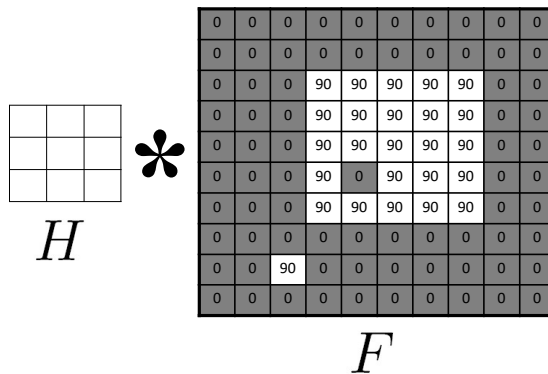
Linear filters: examples



mode='full' con zero padding

88 of 119

Mean filtering



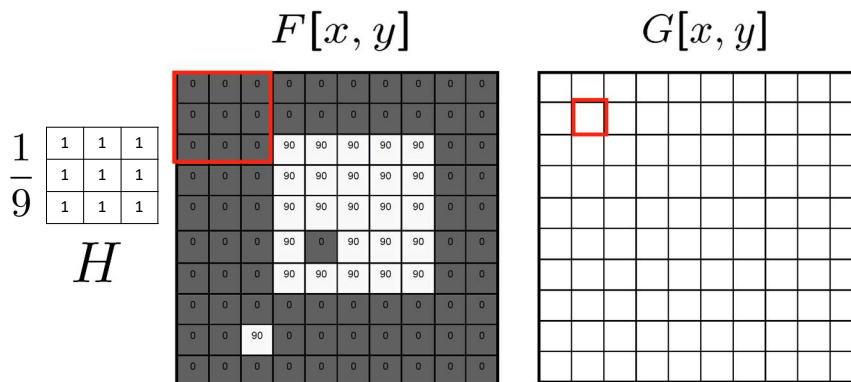
89 of 119

Ahora vamos a comenzar a estudiar el filtrado de medias. Seguimos utilizando exactamente la misma terminología que hemos empleado hasta ahora: tenemos una imagen de entrada, que denominamos F , y un kernel o filtro, que denominamos H .

Lo que vamos a hacer es aplicar una convolución entre la imagen y ese kernel. La diferencia ahora no está en la operación matemática, que es la misma que ya conocemos, sino en los valores concretos del kernel. En un filtro de medias, esos valores se eligen de manera que cada píxel de salida sea el promedio de los píxeles de un determinado vecindario de la imagen.

Por tanto, vamos a utilizar todo lo que ya hemos visto sobre convolución, desplazamiento del kernel, tratamiento de los bordes y cálculo del valor de salida, pero aplicado ahora a uno de los filtros de suavizado más sencillos y habituales.

Mean filtering/Moving average



90 of 119

El filtro de medias también se conoce como box filter. En este caso estamos utilizando un kernel de tamaño 3×3 , por lo que tenemos nueve posiciones, todas con el mismo peso.

Como queremos calcular un promedio, cada uno de esos nueve valores debe contribuir de la misma manera al resultado. Por tanto, el kernel H es simplemente una matriz 3×3 de unos multiplicada por $1/9$.

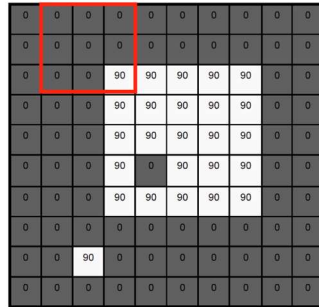
A partir de ahí, desplazamos este kernel por todas las posiciones de la imagen F . En cada posición calculamos la media de los nueve píxeles del vecindario correspondiente y utilizamos ese valor como nuevo píxel de la imagen de salida G .

Podemos interpretar, por tanto, el filtro de medias como una media móvil bidimensional: cada píxel de la nueva imagen se sustituye por el promedio de los valores de su entorno local.

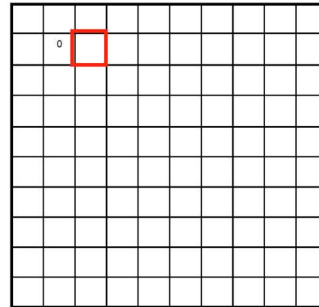
Mean filtering/Moving average

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} H$$

$F[x, y]$



$G[x, y]$



Mean filtering/Moving average

$$\frac{1}{9} H$$

1	1	1
1	1	1
1	1	1

H

$F[x, y]$

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0
0	0	0	90	90	90	90	90	0
0	0	0	90	90	90	90	90	0
0	0	0	90	90	90	90	90	0
0	0	0	90	90	90	90	90	0
0	0	0	90	90	90	90	90	0
0	0	0	90	90	90	90	90	0

$G[x, y]$

	0	10						

Mean filtering/Moving average

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} H$$

$F[x, y]$

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0
0	0	0	90	90	90	90	90	0
0	0	0	90	90	90	90	90	0
0	0	0	90	0	90	90	90	0
0	0	0	90	90	90	90	90	0
0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

$G[x, y]$

	0	10	20					

Mean filtering/Moving average

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

H

$F[x, y]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$G[x, y]$

	0	10	20	30					

Mean filtering

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} * \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 90 & 90 & 90 & 90 & 90 & 0 & 0 \\ 0 & 0 & 0 & 90 & 90 & 90 & 90 & 90 & 0 & 0 \\ 0 & 0 & 0 & 90 & 90 & 90 & 90 & 90 & 0 & 0 \\ 0 & 0 & 0 & 90 & 0 & 90 & 90 & 90 & 0 & 0 \\ 0 & 0 & 0 & 90 & 90 & 90 & 90 & 90 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 90 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} & & & & & & & & & \\ & 0 & 10 & 20 & 30 & 30 & 30 & 20 & 10 & \\ & 0 & 20 & 40 & 60 & 60 & 60 & 40 & 20 & \\ & 0 & 30 & 60 & 90 & 90 & 90 & 60 & 30 & \\ & 0 & 30 & 50 & 80 & 80 & 90 & 60 & 30 & \\ & 0 & 30 & 50 & 80 & 80 & 90 & 60 & 30 & \\ & 0 & 20 & 30 & 50 & 50 & 60 & 40 & 20 & \\ 10 & 20 & 30 & 30 & 30 & 30 & 20 & 10 & & \\ 10 & 10 & 10 & 0 & 0 & 0 & 0 & 0 & & \end{bmatrix}$$

H F G

$\frac{1 \cdot 90}{9} + \frac{1 \cdot 90}{9} = 10 + 10 = 20$

Mean filtering/Moving average

What happens if we don't divide by 9?? 🤔

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} H$$

$F[x, y]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0

$G[x, y]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

Why is the output image smaller than the input image? 🤔

Would there be any difference if we applied correlation instead of convolution? 🤔

What does it do?

- Replaces each pixel with an average of its neighborhood
- Achieve smoothing effect (remove sharp features)

96 of 119

En términos prácticos, lo que estamos haciendo con este filtro es suavizar la imagen. Los valores extremos o muy diferentes de su entorno se promedian con los píxeles vecinos y, por tanto, pierden intensidad. Por ejemplo, si tenemos un valor 90 rodeado de ceros, al aplicar un filtro de medias 3x3 ese valor pasa a contribuir como $90/9 = 10$ en la posición correspondiente de la salida.

El efecto global es que reducimos variaciones locales bruscas y eliminamos detalles finos. La imagen se vuelve más suave, aunque a cambio también podemos difuminar bordes y estructuras pequeñas que sí contenían información relevante.

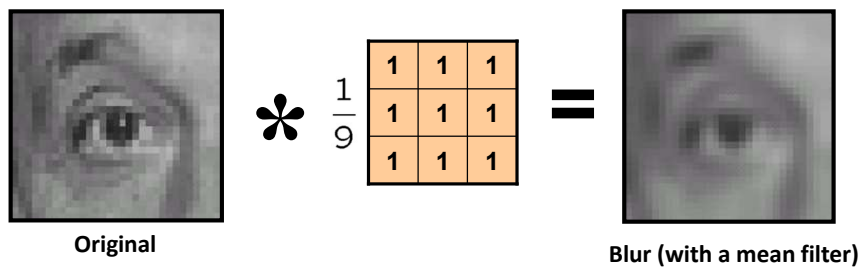
En este ejemplo concreto estamos trabajando en modo valid. Esto significa que solamente calculamos la salida en aquellas posiciones en las que el kernel 3x3 cabe completamente dentro de la imagen, sin utilizar ningún tipo de padding. Como consecuencia, el tamaño de la salida se reduce.

Si la imagen de entrada es de 10x10 y el kernel es de 3x3, el tamaño de la salida será:

$$10 - 3 + 1 = 8$$

por cada dimensión. Por tanto, obtenemos una imagen de salida de 8x8.

Linear filters: examples



“box filter” (all mask values are equal): linear filter in which each pixel in the resulting image has a value equal to the average value of its neighboring pixels in the input image

Source: D. Lowe

97 of 119

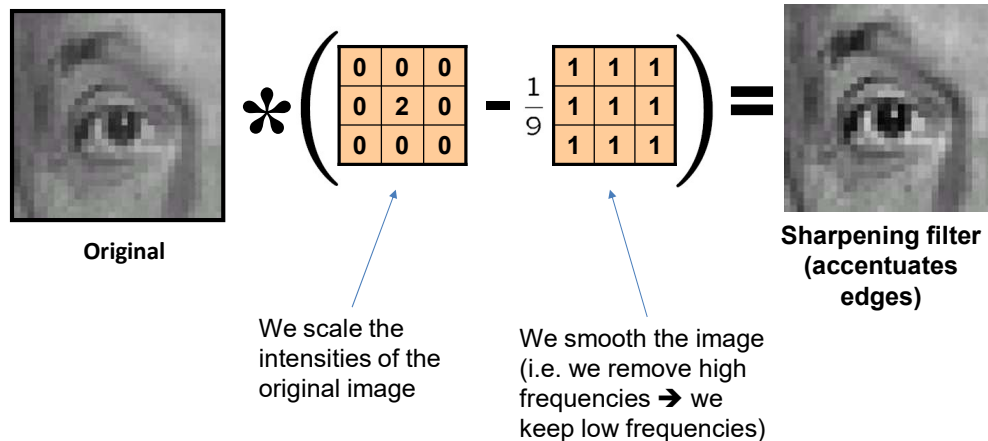
En esta diapositiva aplicamos exactamente el mismo box filter a la imagen del ojo de Albert Einstein.

Como era de esperar, el resultado es una imagen más suavizada o difuminada. Cada píxel de salida se obtiene promediando los valores de un vecindario local, de manera que las variaciones bruscas de intensidad se reducen.

Como consecuencia, desaparecen parte de los detalles finos de la imagen y los bordes se vuelven menos definidos. En este caso concreto, el ojo se percibe claramente más borroso que en la imagen original.

Este ejemplo permite visualizar de forma muy directa el efecto de un filtro de medias: reduce las variaciones locales y suaviza la imagen, aunque a costa de perder detalle.

Linear filters: examples



Source: D. Lowe

98 of 119

El filtrado lineal no sirve solamente para reducir ruido o suavizar una imagen. También puede utilizarse para hacer justamente lo contrario en cierto sentido: realzar detalles, aumentar la nitidez y reforzar los bordes.

Eso es lo que vemos en este ejemplo. Por un lado tenemos un kernel 3x3 con un valor 2 en el centro y ceros en el resto. Aplicarlo a la imagen equivale simplemente a multiplicar la imagen original por 2. Por otro lado tenemos el box filter 3x3 que acabamos de estudiar, que genera una versión suavizada de la imagen.

Lo que hacemos es restar ambos efectos. Intuitivamente, al restar una versión suavizada de la imagen estamos eliminando parte de las componentes de baja frecuencia y conservando o reforzando las variaciones rápidas de intensidad, que son precisamente las asociadas a detalles finos y bordes.

Además, este ejemplo nos permite recuperar una de las propiedades algebraicas de la convolución que vimos anteriormente: la distributividad con respecto a la suma y la resta.

Podemos construir directamente un único kernel restando ambos kernels y convolucionar la imagen una sola vez con ese kernel resultante. Pero también podríamos convolucionar primero la imagen con el kernel que tiene un 2 en el centro, convolucionarla por separado con el box filter y, finalmente, restar ambas imágenes resultantes. El resultado sería

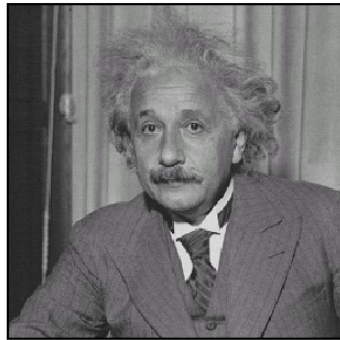
exactamente el mismo.

Es decir:

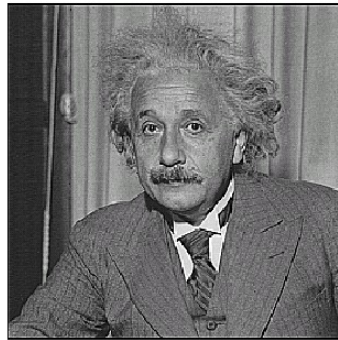
$$F * (H1 - H2) = F * H1 - F * H2$$

Por tanto, esta diapositiva no solo muestra un ejemplo de sharpening/realce, sino también una aplicación directa de la propiedad distributiva de la convolución.

Sharpening



before



after

Source image



Sharpened Image



Highly sharpened image



https://en.m.wikipedia.org/wiki/Unsharp_masking#/media/File%3AUnsharped_eye.jpg

Sharpening amplifies the high-frequency components of the image.

99 of 119

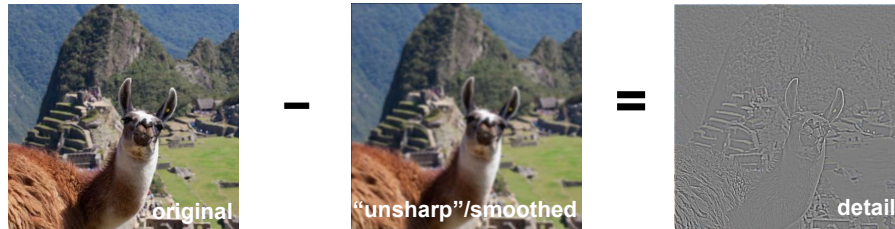
A la izquierda aparece la imagen original de Einstein y, a continuación, una versión realzada. La diferencia principal es que en la imagen procesada los contornos, las texturas y los pequeños detalles aparecen más marcados. En otras palabras, la imagen da una mayor sensación de nitidez.

A la derecha vemos el mismo efecto de forma todavía más clara sobre una imagen de un ojo. Partimos de la imagen original (arriba), aplicamos un cierto grado de sharpening (centro) y finalmente mostramos una versión con un realce mucho más intenso (abajo). A medida que aumentamos el sharpening, los bordes y los detalles locales se hacen cada vez más visibles.

Unsharp Masking

- What does blurring take away?

Unsharp masking is an image sharpening technique



(This "detail extraction" operation is also called a *high-pass filter*)

Let's add it back:

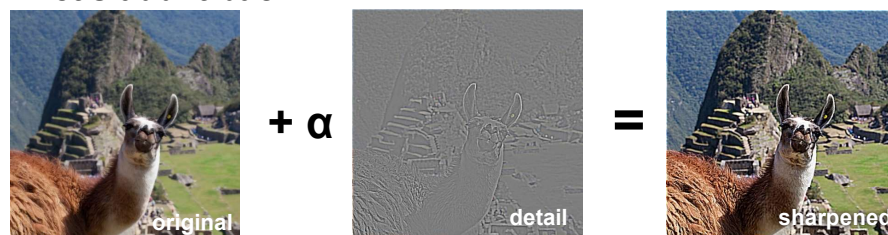


Photo credit: <https://www.flickr.com/photos/geezaweezer/16089096376/>

100 of 119

En esta diapositiva vemos con un poco más de detalle qué ocurre durante el proceso de sharpening mediante unsharp masking.

Partimos de la imagen original y le restamos una versión suavizada de esa misma imagen. La idea intuitiva es que la imagen suavizada conserva principalmente las componentes de baja frecuencia, mientras que las variaciones rápidas de intensidad, asociadas a bordes y detalles finos, quedan atenuadas.

Por tanto, si restamos la imagen suavizada de la imagen original, obtenemos fundamentalmente las componentes de alta frecuencia, es decir, una imagen que contiene sobre todo los detalles que el suavizado había eliminado. Esa es precisamente la imagen que vemos en la parte superior derecha. Como regla mnemotécnica podemos asumir que una imagen (I) consta de la suma de altas (A) y bajas (B) frecuencias: $I = A + B$. Si a la imagen original le restamos las bajas frecuencias nos quedan ($I - B = A$) las altas frecuencias.

A continuación, multiplicamos esa imagen de detalle por un determinado factor alfa, que controla la intensidad del realce, y la sumamos nuevamente a la imagen original. De esta manera estamos reforzando los detalles, los bordes y las variaciones locales de intensidad presentes en la imagen.

El resultado es la imagen sharpened que aparece en la parte inferior derecha: una versión de la imagen original en la que los detalles aparecen más destacados.

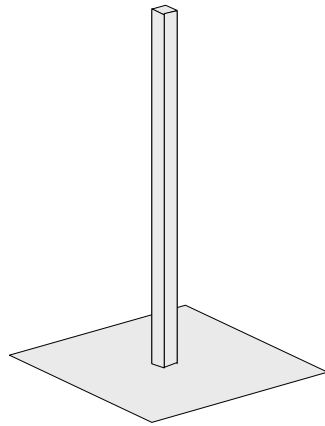
Por tanto, el unsharp masking es una técnica de sharpening basada en una idea

muy sencilla: extraer de la imagen sus componentes de alta frecuencia y volver a sumarmas, amplificadas, a la imagen original.

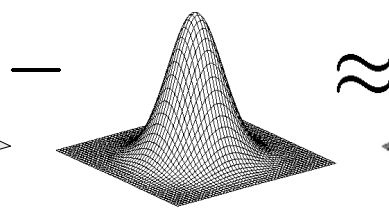
Unsharp Masking

$$F + \alpha (F - F * H) = (1 + \alpha) F - \alpha (F * H) = F * ([1 + \alpha] e - \alpha H)$$

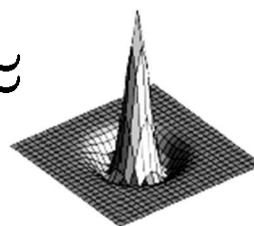
↑ image unsharp / blurred image ↑ unit impulse (identity kernel with single 1 in center, zeros elsewhere)



Scaled impulse



Scaled Gaussian



Sharpen filter

Take-home message: sharpening is taking an image F :

- 1) blurring it,
- 2) subtracting the blurred image from F to get high-frequency details,
- 3) then scaling and adding those edges back to F to yield an image with emphasized edges.

101 of 119

En esta diapositiva profundizamos un poco más en la formulación matemática del unsharp masking. Si F es nuestra imagen original y H es el filtro de suavizado, comenzamos exactamente con la expresión que vimos en la diapositiva anterior: a la imagen original le sumamos una versión escalada de los detalles que obtenemos al restarle su versión suavizada.

Es decir, partimos de $F + \alpha(F - F * H)$. Si desarrollamos la expresión, obtenemos $F + \alpha F - \alpha(F * H)$, que podemos escribir como $(1 + \alpha)F - \alpha(F * H)$.

Ahora utilizamos una propiedad que ya hemos visto anteriormente. La imagen F puede escribirse como la convolución de F con el impulso unidad e , ya que el impulso es el elemento identidad de la convolución: $F * e = F$. Por tanto, $(1 + \alpha)F$ puede expresarse como $F * ((1 + \alpha)e)$, dado que $(1 + \alpha)F = (1 + \alpha)(F * e)$. Utilizando además la distributividad de la convolución, podemos reunir ambos términos y obtener un único filtro equivalente:

$$F * ((1 + \alpha)e - \alpha H)$$

Las tres expresiones que aparecen en la parte superior de la diapositiva son, por tanto, matemáticamente equivalentes. La primera expresa directamente la idea intuitiva del unsharp masking, mientras que la última nos muestra que todo el proceso puede implementarse mediante una

única convolución con un filtro de realce.

Si H es, por ejemplo, un filtro gaussiano, ese filtro de realce se obtiene restando una Gaussiana escalada de un impulso unidad también escalado. Por eso, en la parte inferior de la diapositiva vemos un pico positivo muy intenso en el centro rodeado de valores negativos: es la forma característica de este tipo de filtro de sharpening.

El mensaje fundamental global sigue siendo el mismo. Tomamos una imagen F , la suavizamos, restamos esa versión suavizada de la imagen original para extraer principalmente los detalles de alta frecuencia, multiplicamos esos detalles por un factor α que controla la intensidad del realce y finalmente los añadimos de nuevo a la imagen original. El resultado es una imagen en la que los bordes y los detalles aparecen más marcados.

Unsharp Masking (technical note)

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def unsharp_masking_gaussian_filter(image, gaussianFilter_size=3, amount=1.0):
    # If image has 3 channels, we can convert it to grayscale
    if len(image.shape) == 3:
        gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    else:
        gray_image = image

    # We apply Gaussian smoothing
    smoothed_image = cv2.GaussianBlur(gray_image, (gaussianFilter_size, gaussianFilter_size), 0)

    # We subtract the smoothed image to the original image to get the high-pass filtered image
    high_pass = cv2.subtract(gray_image, smoothed_image)

    # We add the details (high-pass) to the original image
    # dst = cv2.addWeighted(src1, alpha, src2, beta, gamma) --> dst = alpha*src1 + beta*src2 + gamma
    sharp_image = cv2.addWeighted(gray_image, 1.0, high_pass, amount, 0.0)
    # = sharp_image = 1.0*gray_image + amount*high_pass + 0.0

    return gray_image, smoothed_image, high_pass, sharp_image

# Read image
image = cv2.imread('einstein.bmp', 0).astype(np.float64)

# Apply unsharp masking
gray_image, smoothed_image, high_pass, sharp_image = unsharp_masking_gaussian_filter(image, gaussianFilter_size=15, amount=1)
```

102 of 119

En esta diapositiva vemos una implementación directa del unsharp masking utilizando OpenCV. La idea del código reproduce exactamente el procedimiento que acabamos de describir desde el punto de vista matemático.

En primer lugar, recibimos una imagen y comprobamos si tiene tres canales. En ese caso la convertimos a escala de grises; si ya es una imagen monocanal, trabajamos directamente con ella.

A continuación, generamos una versión suavizada mediante GaussianBlur. El parámetro gaussianFilter_size determina el tamaño del kernel gaussiano utilizado para el suavizado. En este ejemplo utilizamos un kernel de 15x15. El valor cero utilizado como sigma indica a OpenCV que estime automáticamente la desviación estándar a partir del tamaño del kernel.

Después calculamos la imagen de detalle restando la imagen suavizada de la imagen original. Conceptualmente estamos calculando:

detalle = imagen original - imagen suavizada

Esta diferencia contiene fundamentalmente las variaciones de alta frecuencia que el filtro gaussiano ha eliminado: bordes y detalles finos.

Finalmente, utilizamos addWeighted para volver a sumar esos detalles a la imagen original. El parámetro amount desempeña el papel del factor

alfa que vimos anteriormente y controla cuánto realce queremos aplicar. Con amount igual a 1 estamos añadiendo una vez la imagen de detalle a la imagen original.

Por tanto, el código implementa directamente:

$\text{imagen realzada} = \text{imagen original} + \text{alfa} \times \text{detalle}$

donde:

$\text{detalle} = \text{imagen original} - \text{imagen suavizada}$

Es importante experimentar con gaussianFilter_size y amount para observar cómo cambia el resultado. Un kernel gaussiano mayor elimina estructuras de mayor escala al construir la imagen suavizada, mientras que un valor mayor de amount refuerza con mayor intensidad los detalles recuperados.

Unsharp Masking



Edges are transformed so that the image gets darker on one side and brighter on the other – in this way, the edge is emphasized. **The sharpened image “anticipates” the edges**

103 of 119

En esta diapositiva vemos de una forma muy visual qué ocurre alrededor de los bordes cuando aplicamos unsharp masking.

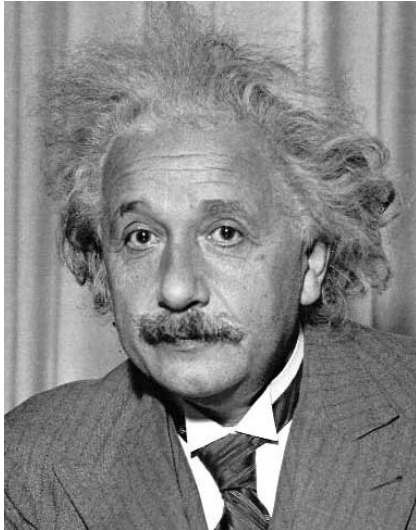
En la parte superior aparece la imagen original, formada por regiones de intensidad aproximadamente constante separadas por bordes relativamente bruscos. En la parte inferior vemos el resultado después del realce.

El sharpening aumenta el contraste local alrededor de la transición. En un lado del borde aparecen valores algo más oscuros y en el otro valores algo más claros. De esta manera, la diferencia de intensidad a ambos lados del borde se hace más pronunciada y nuestro sistema visual percibe el contorno como más nítido.

Este comportamiento puede entenderse a partir del unsharp masking. La diferencia entre la imagen original y su versión suavizada es especialmente grande precisamente cerca de los bordes. Al multiplicar esa componente de detalle por un determinado factor y volver a sumarla a la imagen original, reforzamos esas transiciones.

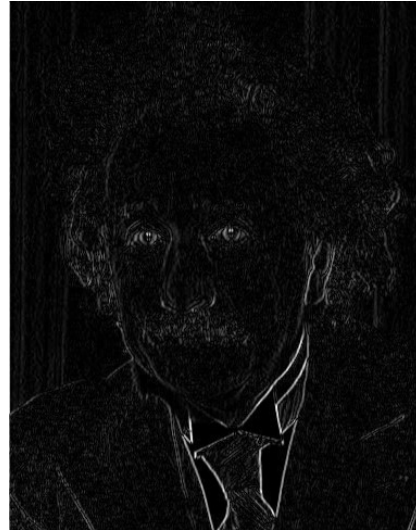
Este mecanismo es precisamente el que genera la sensación de mayor nitidez. El sharpening no crea nuevos detalles reales en la imagen: refuerza las variaciones locales de intensidad que ya estaban presentes, especialmente alrededor de los bordes.

Other filters



1	0	-1
2	0	-2
1	0	-1

Sobel



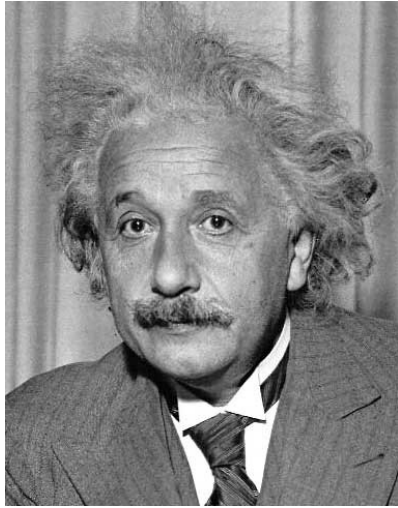
**Vertical Edge
(absolute value)**

104 of 119

Existen muchos filtros lineales diferentes, y uno de los aspectos fundamentales que debemos aprender es que el comportamiento del filtro viene determinado por los coeficientes del kernel. Dependiendo de los valores que coloquemos en la máscara, estaremos realizando una operación diferente sobre la imagen: suavizado, realce, detección de bordes, etc.

En esta diapositiva vemos un ejemplo clásico: un kernel de Sobel de 3x3 diseñado para responder a variaciones de intensidad en la dirección horizontal. Dicho de otra forma, aproxima la derivada de la imagen respecto a la coordenada horizontal y, por tanto, responde especialmente bien ante bordes verticales. Este tipo de filtros los veremos con más calma en próximas sesiones.

Other filters



1	2	1
0	0	0
-1	-2	-1

Sobel



Horizontal Edge
(absolute value)

105 of 119

En esta diapositiva vemos el caso complementario: un kernel de Sobel 3x3 para detectar bordes horizontales.

En este caso, el kernel aproxima la derivada de la imagen respecto a la coordenada vertical. Por tanto, responde especialmente bien cuando existe un cambio brusco de intensidad de arriba hacia abajo, que es precisamente lo que caracteriza a un borde horizontal.

Image Filtering (key ideas so far)

- Convolution is a **local and linear operation**
 - Red numbers below represent the coefficients of the mask.
 - The filter is multiplied element-by-element with the image, the products are added, and the central position of the filter is replaced in the image.



<https://glassboxmedicine.com/2019/07/26/convolution-vs-cross-correlation/>

106 of 119

Llegados a este punto, y a modo de resumen, podemos recapitular algunas de las principales ideas que hemos visto hasta ahora.

La convolución es una operación lineal y local que calcula cada píxel de salida a partir de un vecindario de la imagen. En cada posición desplazamos el kernel sobre la imagen y calculamos el producto escalar entre los coeficientes del kernel y los píxeles del vecindario correspondiente.

En esta diapositiva, los números en rojo representan precisamente los coeficientes de la máscara, kernel o filtro. Cada coeficiente se multiplica por el píxel de la imagen sobre el que se encuentra y, a continuación, sumamos todos esos productos.

El valor resultante se asigna a la posición de salida correspondiente al centro del kernel. Después desplazamos el kernel a la siguiente posición y repetimos exactamente el mismo procedimiento hasta recorrer la imagen.

Por tanto, la idea fundamental es muy sencilla: multiplicación elemento a elemento, suma de los productos y asignación del resultado al píxel correspondiente de la imagen de salida.

Image Filtering (key ideas so far)

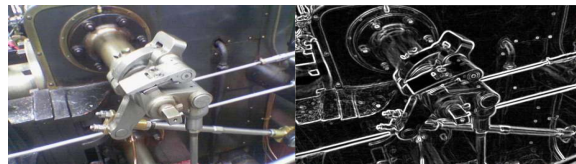
- Convolution is a **local and linear operation**
 - The coefficients of the mask determine the result

Gaussian filter (attenuates high frequencies → image smoothing)

$$\frac{1}{273} \begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}$$


Sobel filter (attenuates low frequencies → enhances edges/boundaries)

$$\mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} \quad \mathbf{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$



107 of 119

La tarea que queremos resolver es, en último término, la que determina los coeficientes concretos del kernel. Dependiendo de esos valores, el filtro producirá un efecto u otro sobre la imagen.

Por ejemplo, en esta diapositiva vemos un kernel cuyos coeficientes siguen aproximadamente una distribución gaussiana. Este tipo de filtro atenúa las componentes de alta frecuencia y, como consecuencia, suaviza la imagen. Recordemos que las altas frecuencias están asociadas a variaciones rápidas de intensidad, como detalles finos, ruido y bordes.

En la parte inferior vemos un filtro de Sobel. En este caso, los coeficientes están diseñados para aproximar una derivada espacial. Por ello, el filtro responde muy poco en regiones de intensidad aproximadamente constante y produce valores elevados allí donde existen cambios bruscos de intensidad, es decir, principalmente en los bordes.

Debemos entender que ambos kernels están diseñados para objetivos distintos: la Gaussiana suaviza y atenúa las variaciones rápidas, mientras que Sobel destaca los cambios bruscos de intensidad.

Image Filtering

- Image kernels visually explained
 - <https://setosa.io/ev/image-kernels/>

Let's walk through applying the following 3x3 right sobel kernel to the image of a face from above.

right sobel

$$\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

Below, for each 3x3 block of pixels in the image on the left, we multiply each pixel by the corresponding entry of the kernel and then take the sum. That sum becomes a new pixel in the image on the right. Hover over a pixel on either image to see how its value is computed.



108 of 119

En esta diapositiva podéis ver una URL que puede resultar muy útil para desarrollar una intuición visual sobre cómo funciona el filtrado mediante kernels.

La herramienta muestra una imagen de entrada y permite seleccionar distintos filtros. Para cada uno de ellos podéis observar el kernel sobre la imagen, qué vecindario local se está utilizando en cada momento y qué operación concreta se realiza sobre sus píxeles.

Además, permite desglosar el cálculo del producto escalar entre el kernel y el vecindario correspondiente, así como ver el valor que finalmente se asigna a cada píxel de la imagen de salida.

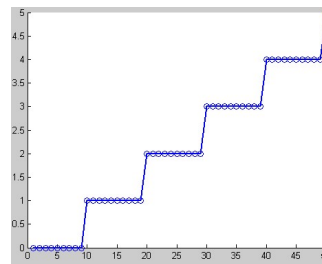
Este tipo de recursos interactivos son especialmente útiles para comprender y asentar las ideas fundamentales del procesamiento de imagen. Permiten visualizar qué está ocurriendo realmente en cada posición y relacionar los coeficientes del kernel con el efecto que producen sobre la imagen.

Nonlinear filters

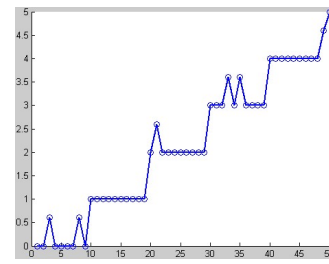
Median filter: Replace pixel with median of surrounding $n \times n$ region (no new pixel values introduced)

Good for impulse noise (salt-and-pepper noise)

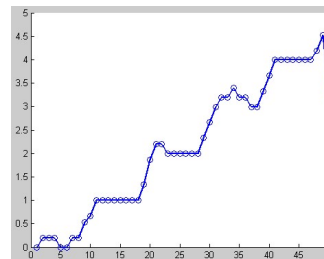
What is a mean filter good for?
Additive Gaussian noise



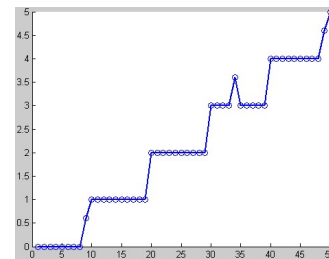
image



impulse noise



mean filtering



median filtering

S. Birchfield, Clemson Univ., ECE 847, <http://www.ces.clemson.edu/~stb/ece847>

109 of 119

Hasta ahora hemos hablado fundamentalmente de filtros lineales, pero también existen filtros no lineales. Uno de los ejemplos más importantes es el filtro de mediana.

En este caso ya no calculamos una combinación lineal de los valores del vecindario. Para cada posición, tomamos los píxeles de una región local de tamaño $n \times n$, los ordenamos según su intensidad y sustituimos el píxel correspondiente de la imagen de salida por la mediana de esos valores.

Si utilizamos un vecindario con un número impar de elementos, como es habitual (3x3, 5x5, 7x7, etc.), la mediana coincide con uno de los valores presentes en el propio vecindario. Por tanto, a diferencia de un filtro de medias, no generamos un nuevo valor mediante un promedio, sino que seleccionamos uno de los valores existentes.

El filtro de mediana también produce un efecto de suavizado y reducción de ruido, pero tiene un comportamiento diferente al de los filtros lineales. Es especialmente eficaz frente al ruido de tipo sal y pimienta. Un píxel extremadamente claro u oscuro puede ser eliminado porque, al ordenar los valores del vecindario, ese valor extremo normalmente no ocupará la posición mediana.

Por otro lado, frente al ruido aditivo gaussiano, tanto los *box filters* como los filtros gaussianos pueden resultar efectivos. Al combinar los valores de varios píxeles vecinos, las fluctuaciones aleatorias positivas y negativas del ruido tienden a compensarse, reduciendo su efecto.

El mensaje clave es que no existe un único filtro adecuado para cualquier tipo de ruido: el modelo de ruido presente en la imagen condiciona qué estrategia de filtrado resulta más apropiada.

Nonlinear filters

Image with salt-and-pepper noise



mean filtering (3x3)



mean filtering (11x11)



Why does a dark border appear when using the 11×11 mean filter?

https://cs.brown.edu/courses/cs143/2017_Spring/lectures_Spring2017/04_Spring2017_ThinkingInFrequencyPart1.pptx

110 of 119

Tanto en esta diapositiva como en la siguiente vemos una imagen contaminada con ruido sal y pimienta y comparamos distintos métodos de filtrado.

En esta primera diapositiva aplicamos un filtro de medias con dos tamaños diferentes: 3x3 y 11x11. Con el filtro 3x3 conseguimos cierto suavizado, pero el ruido impulsivo no se elimina especialmente bien, porque los valores extremos introducidos por el ruido siguen influyendo directamente en el promedio del vecindario.

Si aumentamos el tamaño del filtro hasta 11x11, el efecto de suavizado es mucho más intenso y parte del ruido se atenúa en mayor medida. Sin embargo, el precio que pagamos es una pérdida importante de detalle: la imagen aparece mucho más difuminada y los contornos de los objetos se degradan considerablemente.

Nonlinear filters

Image with salt-and-pepper noise



median filtering (3x3)



median filtering (11x11)



Why does the dark border not appear when using the 11×11 median filter?

https://cs.brown.edu/courses/cs143/2017_Spring/lectures_Spring2017/04_Spring2017_ThinkingInFrequencyPart1.pptx

111 of 119

Ahora aplicamos un filtro de mediana sobre la misma imagen contaminada con ruido sal y pimienta, utilizando dos tamaños de vecindario diferentes: 3x3 y 11x11.

Con el filtro de mediana 3x3 el resultado es muy notable. El ruido sal y pimienta se elimina en gran medida porque los valores extremos introducidos por este tipo de ruido dejan de dominar el resultado. Al ordenar los valores del vecindario y escoger la mediana, esos píxeles anómalos suelen quedar en los extremos y no son seleccionados.

Cuando aumentamos el tamaño del filtro hasta 11x11, el ruido sigue eliminándose eficazmente, pero aparece otro efecto: el filtrado empieza a ser excesivamente agresivo y se pierden estructuras y detalles de la imagen.

Visualmente, la imagen deja de tener el aspecto natural de una fotografía y adquiere una apariencia más simplificada, casi pictórica, como si hubiese sido pintada. Esto ocurre porque un vecindario tan grande hace que muchas pequeñas variaciones y texturas locales sean sustituidas por valores medianos representativos de regiones más amplias.

Thresholding



$$g(m, n) = \begin{cases} 255, & f(m, n) > A \\ 0 & \text{otherwise} \end{cases}$$

112 of 119

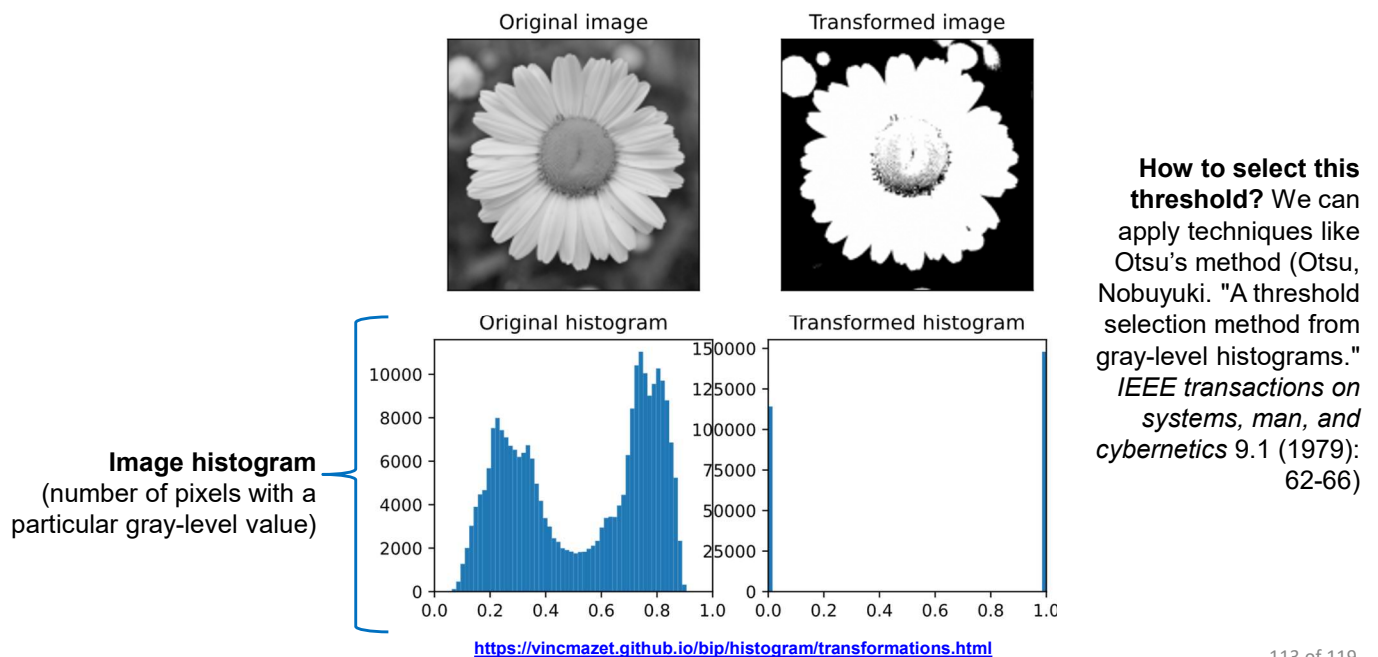
Para terminar la presentación, vamos a introducir dos conceptos muy sencillos pero muy útiles: el histograma de una imagen y el umbralizado o thresholding.

El umbralizado consiste en seleccionar un determinado valor de intensidad y utilizarlo para dividir los píxeles de la imagen en dos grupos. Por ejemplo, podemos asignar el valor 255, correspondiente al blanco, a todos los píxeles cuya intensidad sea superior al umbral, y el valor 0, correspondiente al negro, a los que sean menores o iguales. El resultado es una imagen binaria, como podemos observar en el ejemplo de la Gioconda mostrado en la diapositiva.

El umbralizado puede utilizarse como una técnica muy sencilla de segmentación. La idea es separar regiones de interés, por ejemplo un objeto o primer plano, del fondo utilizando únicamente sus diferencias de intensidad.

A pesar de su simplicidad, el thresholding sigue siendo útil en numerosas aplicaciones prácticas, especialmente cuando existe una separación suficientemente clara entre las intensidades correspondientes a las distintas regiones que queremos distinguir.

Thresholding



113 of 119

En esta diapositiva vemos también el histograma de una imagen. En la parte superior izquierda aparece la imagen original de una flor y, justo debajo, su histograma.

El histograma es una representación gráfica de la distribución de intensidades de la imagen. En el eje horizontal aparecen los distintos niveles de gris. En este ejemplo están normalizados entre 0 y 1, de manera que 0 corresponde al negro y 1 corresponde al valor 255 al blanco (en una imagen de 8 bits). En el eje vertical se representa cuántos píxeles, o con qué frecuencia, aparece cada nivel de intensidad.

En este caso, observamos un histograma aproximadamente bimodal, es decir, aparecen dos grupos principales de intensidades. Esta situación puede ser adecuada para aplicar un umbral, ya que podemos escoger un valor que separe ambos grupos. Sin embargo, esta separación se realiza únicamente en función de la intensidad y no garantiza una separación semánticamente relevante.

Al aplicar el thresholding, cada píxel pasa a pertenecer únicamente a una de dos categorías: negro o blanco. El resultado es la imagen binaria que vemos en la parte superior derecha.

Y aquí aparece una de las preguntas fundamentales: ¿cómo seleccionamos el valor del umbral? En algunos casos puede elegirse manualmente a partir del conocimiento de la aplicación o incluso observando el histograma. Sin embargo, también existen métodos que permiten determinarlo automáticamente a partir de la

distribución de intensidades de la propia imagen.

Uno de los métodos clásicos y más conocidos para realizar esta selección automática es el método de Otsu, que veremos en la siguiente diapositiva.

Otsu's Thresholding Method

We **iterate through all possible threshold values** and **measure the spread/variance** of background and foreground pixels. Then, **we select the threshold with the smallest spread/variance**. i.e. we want to minimize the within-class variance: $\sigma^2(t) = \omega_{bg}(t)\sigma_{bg}^2(t) + \omega_{fg}(t)\sigma_{fg}^2(t)$

Let's try with this image and threshold (t=100):

Foreground Pixels

Background Pixels

120	120	21	22
25	26	27	160
180	190	123	145
165	175	23	24

$$\begin{aligned} P_{all} &= 16 \\ P_{BG} &= 7 \\ P_{FG} &= 9 \end{aligned}$$

$$\begin{aligned} \omega_{bg}(t) &= \frac{P_{BG}(t)}{P_{all}} = 7/16 = 0.44 \\ \omega_{fg}(t) &= \frac{P_{FG}(t)}{P_{all}} = 9/16 = 0.56 \end{aligned}$$

$$\begin{aligned} \bar{x}_{bg} &= \frac{21+22+25+26+27+23+24}{7} = 24 \\ \bar{x}_{fg} &= \frac{120+120+160+180+190+123+145+165+175}{9} = 153.1 \end{aligned}$$

$$\begin{aligned} \sigma_{bg}^2(t=100) &= \frac{(21-24)^2 + (22-24)^2 + \dots + (24-24)^2}{7} = 4.0 \\ \sigma_{fg}^2(t=100) &= \frac{(120-153.1)^2 + (120-153.1)^2 + \dots + (175-153.1)^2}{9} = 657.43 \end{aligned}$$

$$\sigma^2(t=100) = 0.44 * 4.0 + 0.56 * 657.43 = 369.9208$$

We keep the threshold corresponding to the smallest σ^2
We want two clusters that are very cohesive internally (low variance)

Otsu, Nobuyuki. "A threshold selection method from gray-level histograms."
 IEEE transactions on systems, man, and cybernetics 9.1 (1979): 62-66

<https://muthu.co/otsus-method-for-image-thresholding-explained-and-implemented/>

El método de Otsu, publicado en 1979, es una técnica automática para seleccionar el umbral de una imagen a partir de su distribución de intensidades. La idea consiste en evaluar los posibles valores de threshold y, para cada uno de ellos, dividir los píxeles en dos clases: background y foreground.

Para cada umbral calculamos el peso o proporción de píxeles perteneciente a cada clase, la media de intensidad de cada una y su varianza. A continuación, calculamos la varianza intraclase ponderada, es decir, una combinación de las varianzas del background y del foreground teniendo en cuenta el tamaño relativo de cada grupo.

En esta diapositiva vemos un ejemplo para una imagen de 4x4 y un umbral T igual a 100. En primer lugar, tenemos 16 píxeles en total. Con este umbral, 7 píxeles pertenecen al background y 9 al foreground. A partir de ahí calculamos la proporción de píxeles de cada clase, sus intensidades medias y sus correspondientes varianzas. Sustituyendo estos valores en la expresión de la parte superior obtenemos, para T igual a 100, una varianza intraclase de 361,9208.

Este procedimiento se repite para los posibles valores del umbral y finalmente seleccionamos aquel que minimiza la varianza intraclase. La intuición es que queremos obtener dos grupos cuyos píxeles sean lo más homogéneos posible internamente, es decir, dos clases con poca dispersión dentro de cada una.

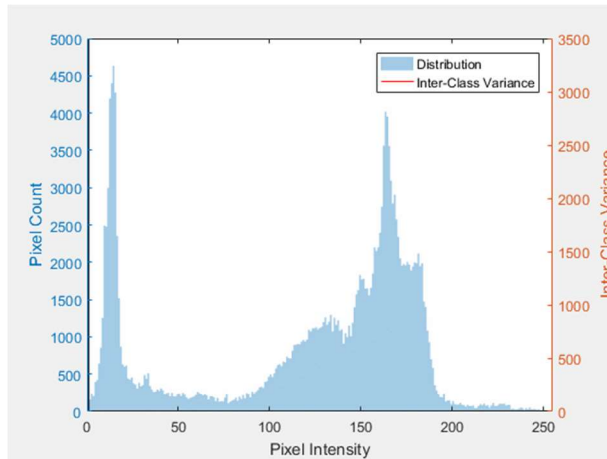
De forma equivalente, el método de Otsu puede interpretarse como la búsqueda del umbral que maximiza la varianza interclase, es decir, que produce la mayor

separación posible entre las medias del background y del foreground.

Por tanto, el objetivo es muy intuitivo: encontrar automáticamente el punto de corte que produzca dos clases internamente compactas y, al mismo tiempo, bien separadas entre sí.

Otsu's Thresholding Method

Minimizing the intra-class variance is equivalent to maximizing inter-class variance



https://en.wikipedia.org/wiki/Otsu%27s_method#/media/File:Otsu's_Method_Visualization.gif



Otsu, Nobuyuki. "A threshold selection method from gray-level histograms."
IEEE transactions on systems, man, and cybernetics 9.1 (1979): 62-66

Check [Ejemplo_Otsu.ipynb](#)
in PRADO!

115 of 119

En esta diapositiva vemos una animación que permite visualizar de forma muy intuitiva cómo funciona el método de Otsu. En azul aparece el histograma de intensidades de la imagen y, a medida que se van probando distintos valores de umbral, se va calculando la varianza interclase correspondiente.

Como ya hemos visto, minimizar la varianza intraclase es equivalente a maximizar la varianza interclase. Por tanto, podemos buscar directamente el valor del umbral para el que la separación entre las dos clases es máxima.

Eso es precisamente lo que representa la curva roja que se va construyendo en la animación. Para cada posible threshold se calcula la varianza interclase y buscamos su valor máximo. La posición de ese máximo determina el umbral seleccionado por el método de Otsu.

A la derecha vemos un ejemplo de aplicación sobre una imagen de edificios. En la parte superior aparece la imagen original y en la parte inferior el resultado de aplicar el umbral calculado automáticamente mediante Otsu. En este caso, la separación obtenida entre las dos clases es visualmente bastante satisfactoria.

Finalmente, la idea de Otsu también puede generalizarse cuando queremos dividir las intensidades en más de dos clases. En lugar de buscar un único umbral que separe background y foreground, podemos buscar varios umbrales que dividan el histograma en múltiples regiones. El objetivo sigue siendo el mismo: encontrar la partición que maximice la separación entre las clases, o equivalentemente que minimice su dispersión interna. Esta generalización se conoce como Multi-Otsu.

Histogram Equalization

We **increase the global contrast**, especially when the image is represented by a narrow range of intensity values.
After equalization, **the intensity values are more evenly distributed across the full range**.

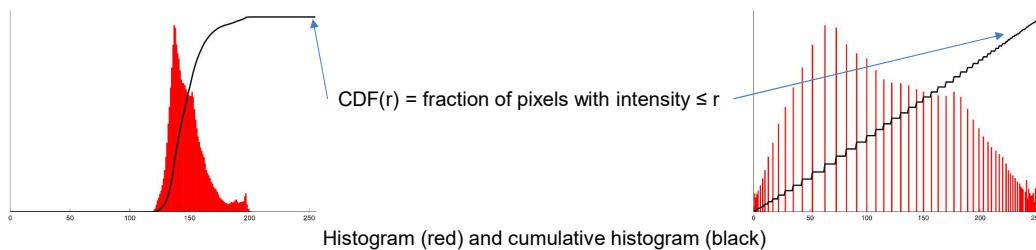


Before

After



input intensity $r \rightarrow$ cumulative probability $CDF(r) \rightarrow$ output intensity $s = 255 \cdot CDF(r)$



116 of 119

En esta diapositiva introducimos el concepto de ecualización del histograma. A la izquierda vemos una imagen cuyo histograma está muy concentrado en una zona relativamente estrecha del rango de intensidades. Esto significa que muchos píxeles presentan niveles de gris bastante similares y, como consecuencia, la imagen tiene poco contraste y puede resultar difícil distinguir determinados detalles.

La ecualización del histograma intenta redistribuir esos niveles de intensidad para aprovechar mejor el rango disponible. Para ello utiliza la distribución acumulada de intensidades (CDF), que es la curva negra que aparece en la diapositiva, y construye una transformación que separa más aquellos niveles de gris que aparecen con mayor frecuencia.

El resultado es un histograma más extendido a lo largo del rango de intensidades y, normalmente, un aumento del contraste global de la imagen. Como podemos observar en la imagen de la derecha, después de la ecualización se distinguen con mayor claridad estructuras y diferencias de intensidad que apenas eran visibles en la imagen original.

Una idea que quiero destacar es que incluso una representación tan simple como el histograma puede proporcionar información suficiente para realizar tareas útiles de procesamiento de imagen.

Por ejemplo, a partir del histograma podemos seleccionar un umbral para separar distintas regiones de la imagen, como ocurre en el thresholding y en métodos automáticos como Otsu. También podemos transformar la distribución de

intensidades para incrementar el contraste global, como hacemos mediante la ecualización del histograma.

Por tanto, aunque el histograma pierde completamente la información espacial de la imagen, sigue siendo una descripción muy útil para determinadas operaciones globales de procesamiento.

General Note

- In computer vision, our results (and our interpretation of them) are affected by two key aspects:
 - **Visualization** (i.e., we perform the correct operations, but our visualization makes us think that we are not doing the correct thing)
 - **Operation** (i.e., we are not performing the right operation (algorithmically or at the implementation level), but our visualization makes us think that we are doing the right thing)
 - This last situation is even more problematic, because we are doing something wrong without realizing it...
- We must be aware of these two dimensions in computer vision!

117 of 119

Un elemento fundamental que tenéis que tener siempre presente en visión por computador es que nuestros resultados y la interpretación que hacemos de ellos pueden verse afectados por dos tipos de problemas diferentes.

Por un lado, podemos tener un problema de visualización. Es decir, es posible que la operación que hemos realizado sea correcta, pero que la forma en la que mostramos el resultado nos lleve a pensar que algo ha salido mal. Esto puede ocurrir, por ejemplo, porque la herramienta de visualización transforma los datos (sin que nosotros seamos conscientes de ello) antes de mostrarlos.

Por otro lado, podemos tener un problema en la propia operación. En ese caso, el algoritmo o la implementación son incorrectos, pero la visualización puede hacer que el resultado parezca razonable y llevarnos a pensar que estamos haciendo las cosas bien. Esta situación es incluso más problemática, porque estamos cometiendo un error sin ser conscientes de ello.

Por tanto, en visión por computador es muy importante mantener siempre una actitud crítica y comprobar continuamente qué estamos calculando y qué estamos visualizando. Una imagen que “parece correcta” no garantiza que la operación lo sea, y una imagen que “parece incorrecta” tampoco implica necesariamente que el cálculo esté mal.

El mensaje clave es que debemos distinguir siempre entre dos niveles: la operación interna que estamos realizando sobre los datos y la manera en la que representamos visualmente el resultado. Ambas pueden introducir errores o interpretaciones engañosas, y debemos comprobarlas por separado.

Homework about linear filters



- Is convolution equivariant to rotation and scaling?
- Is correlation equivalent to convolution if we rotate its output?
In other words, $rotate180(Correlation(I, K)) == Convolution(I, K)$?
- Can thresholding be implemented with a linear filter? In other words, is thresholding a linear operation?

118 of 119

En esta diapositiva y la siguiente tenéis una serie de ejercicios que os recomiendo realizar en casa para comprobar si habéis entendido correctamente algunos de los conceptos fundamentales relacionados con los filtros lineales.

Homework about linear filters



- Which one of the following 3x3 filters returns a positive value if the average value of the 4-adjacent neighbors is less than the center, and a negative value (or zero) otherwise?

(a)

0	$\frac{1}{4}$	0
$\frac{1}{4}$	-1	$\frac{1}{4}$
0	$\frac{1}{4}$	0

(b)

-1	-2	-1
0	0	0
1	2	2

(c)

0	-1	0
-1	4	-1
0	-1	0

Image Representation and Filtering

Pablo Mesejo

pmesejo@go.ugr.es

Universidad de Granada

Departamento de Ciencias de la Computación e Inteligencia Artificial



UNIVERSIDAD
DE GRANADA



DaSCI

Instituto Andaluz de Investigación en
Data Science and Computational Intelligence